

Catalog Sync & Storefront	2
Headless Storefront	13
Storefront Model & Part Page Data Requirements	17
Brand Equipment Type Page	20
Brand Home	21
Product Page	22
Catalog and Legacy Data Ingestion & Sync for MVP	23
BE TEAM TECH DESIGNS	36
Tech Design: Change Stream Consumer	37
Tech Design: EventBridge Bus Service	39
Tech Design: IPL Custom Entity Plugin	44
Tech Design: Lambdas for ongoing sync service	49
[Eliminated] Tech Design: SQS	53
[Eliminated] Tech Design: Lambdas for load service	56
CDN - Approach - [Outdated]	59
Custom Fields	60
List of Custom Fields	61
Custom Fields - Name Convention	72
Cataloger DB to Shopware mapping	73
Data Ingestion Architecture	74
Data Ingestion: Ongoing sync trigger	79
Refactoring MongoDB Data Pipeline for Improved Error handling	84
DocumentDB-Shopware normalization -> validation -> fixing mapping	93
Catalog Hierarchy	99

Catalog Sync & Storefront

Summary

Current State

Proposal for MVP

TBD / Decisions

Sub-System Architecture

MVP

Migration Possibilities (WIP)

Products

Models

IPLs/Parts Diagrams

Platform Implications

Shopware

Summary

Current State

Cataloger (UI, Batch Catalog Manipulation), **Catalog DB** (Local MongoDB) to  **PartsTree - Home of OEM Parts for Outdoor Power Equipment** (AWS DocumentDB)

The first version of the cataloger was local, and developers would modify data offline and then upload it to the on-premises catalog when they returned

The current process involves several key steps:

Data Harvesting:

Data sources used to come physically on CD, and the amount of data was significant enough that they preferred to keep it on-premises. The data transfer would be easier locally. They still receive CDs for most brands, and the process involves ripping data from the CD on a local machine to process it into the catalog.

Cataloger Processing:

The cataloger has several important flags in its system:

- A "sync" flag to specify if a document is scheduled for sync
- A "review" text field where someone importing data can mark something for review
- An "approved" flag that indicates someone has QA'd the data and deemed it correct for the next sync

Data Preparation and Sync:

The process is managed by a CRON job with a scheduled time known to everyone. When the ETL process kicks off:

- It uses metadata filters
- Checks if items are approved
- Checks if items are deleted
- Sends appropriate data to the website DB

Website Integration:

The catalog interfaces with the website through the website's API. The website doesn't have direct knowledge of the cataloger. Everything the catalog does is to send data to the website's API

DocumentDB and Elasticsearch:

- Models don't directly go into DocumentDB like products, but rather go into DocumentDB primarily for the purpose of indexing into Elasticsearch

The purpose of these models is purely informational - for search and fast reading. There's no frequent need to update individual pieces of data.

Additional Nuances:

The cataloger is intentionally designed to be agnostic of the website. It was built so that any additional services can consume its data. The cataloger doesn't care that it's sending data to the website - it's meant to be a flexible data source

Proposal for MVP

Database Location and Access:

The proposal is to move the master catalog (currently a MongoDB) into the AWS VPC for improved accessibility. This would also involve moving the cataloger software that manages the database into the VPC.

Central Data Repository:

The goal is to create a database in the cloud that:

- Parts Tree can load into, update, manage
- Altus Nova can use as a source for Shopware loads and syncs
- Maintains important properties that dictate whether items are ready to update in Shower

Preservation of Existing Processes:

The approved flag would remain a top-level indicator of whether data is correct and ready for

other services to use. Any ingestion service would need to respect this existing approval mechanism.

Approval and Sync Mechanism:

proposing a system with:

- A queue of changes
- Visibility into what changes are queued
- A way to show old vs. new data (like code diffs, but more user-friendly)
- A manual user approval process for safety and execution of changes to Shopware

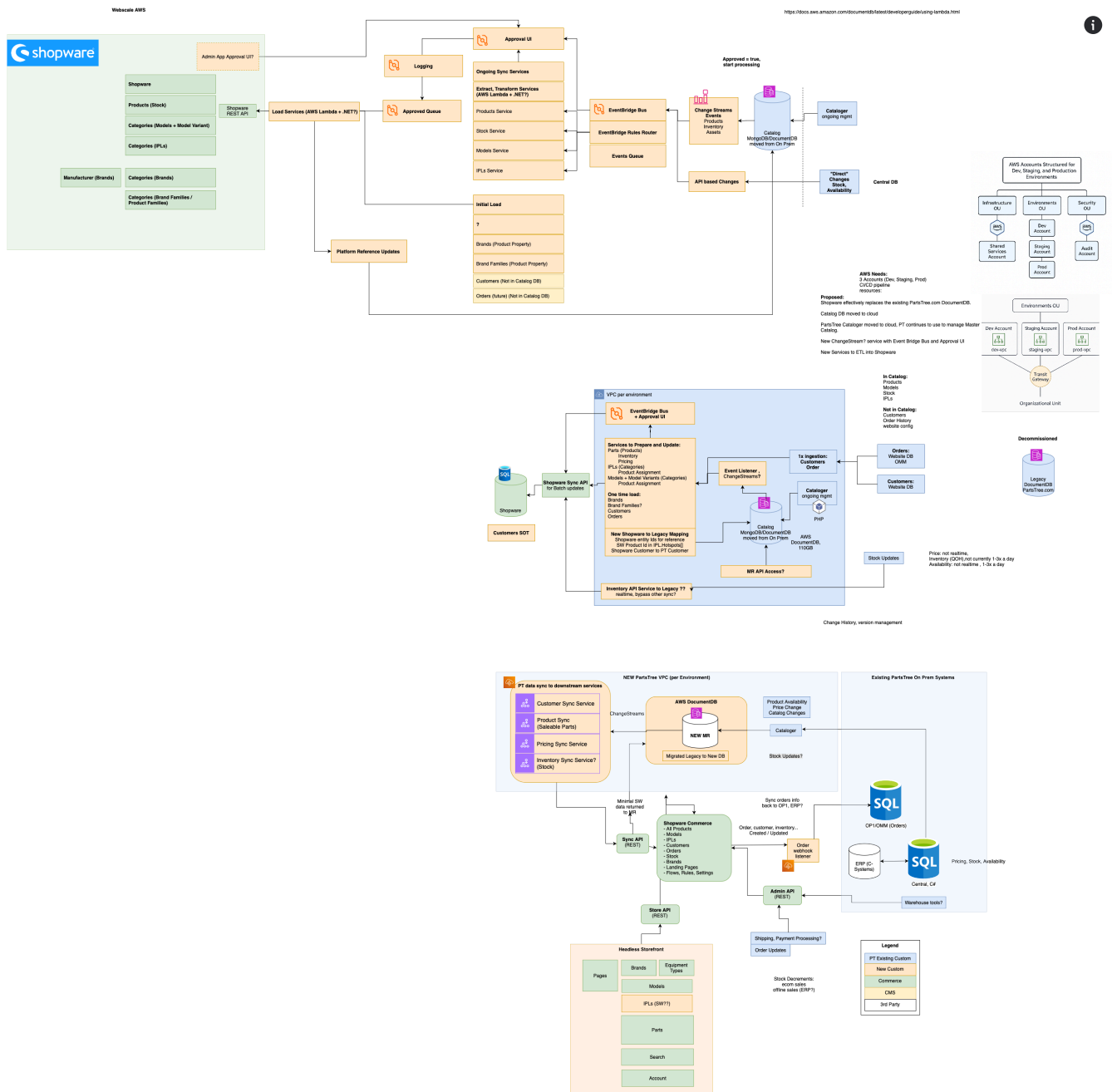
Future Evolution:

Eventually, they might:

- Leverage a PIM (Product Information Management) to replace cataloger functionality
- Rebuild cataloger functionality inside the PIM
- Maintain the core system of services needed for this evolution

Key Principles:

The cataloger was originally built to be service-agnostic. It doesn't care about the website and was specifically designed so that any additional services can consume its data. Altus Nova coming in and using the cataloger is exactly what it was built for.



TBD / Decisions

Question	Answer	Reasoning	Date
Multiple DB sources for new storefront	Expecting all Product Catalog and Page	Models, IPLs, Equipment Types as	

system?	content to come from Shopware Store API	Categories, Parts as Products.	
Syncing Mechanism	Catalog DB → EventBridge?		
Only Saleable Parts into Commerce?	No, All Products	Expecting all Products, Saleable or Not to live in Shopware	
Models in Commerce ?	Yes, as Categories	Shopware approved	
IPLs in Commerce?	Yes, as Categories	Shopware approved	
Supersedes	Custom Field? (Entity Reference)		
Future state cataloger	TBD Post MVP		

Sub-System Architecture

MVP

Migration Possibilities (WIP)

Goal: reducing reliance on maintaining current catalog db

- simplifies search infrastructure?
- helps with part rationalization? part equivalency transparency for customer

Models → IPLs, IPLs → Parts

- Option 1: Model only has some “recommended products” , but has connected IPL (Categories) Custom Field, which combine to hold full list of Model Products (combined IPL subsets)
 - less sync processes to populate parts in both Category sets

- cleaner?
- Option 2: Model has Parts mapped and IPLs have Parts mapped.
 - extra effort in sync process to populate all the values twice?
 - more possibilities for data retrieval

Use Cases:

1. Parts without IPLs (%?)
2. Show and Search all of a Model's Parts
 - a. text search to filter list of parts
 - b. ability to scroll through all (cumbersome if many hundreds of parts per model)
 - c. need to filter by type as well (belts, bolts, etc)
3. Render list of Models that a Part fits
 - a. API call to get product 's IPLs?

i. 1 POST {url}/store-api/product/{productId}?
associations[0]=categories&filter[category.tagIds][0]={tagId}

- b. Product → Categories (filtered by Tag, or Parent, custom field?)

i. /store-api/product/{productId} with body like:

ii.

```

1  {
2    "associations": {
3      "categories": {
4        "associations": {
5          "tags": {}
6        }
7      }
8    },
9    "includes": {
10     "product": ["id", "name", "categories"],
11     "category": ["id", "name", "tags"],
12     "tag": ["id", "name"]
13   }
14 }
```

can be filtered so category.tags.name == "IPL"

Name	Volum e	Catalog Relationship	New Location	Search Impact	Managemen t Impact
Models	300K	Models connect to 10s of IPLs	PartsTree DB? Commerce	External Search Req if not	

		• IPLs can connect to many Models • **New mapping of Parts to Models?? Currently only a few Recommended Parts are connected to Models •	Platform (Categories)	in Commerce	
Model Parent		• Model Parent can have many “Model Variants”	PartsTree DB? Commerce Platform (Categories)		
IPLs / Parts Diagrams	1M	• IPLs can connect to many Models • IPLs connect to 10s of Parts	PartsTree DB? Commerce Platform (Categories)		
Total Products (Parts)	3.5M	• Not currently connected to Models • List of Models not currently an attribute • List of IPLs not currently an attribute	Commerce Platform (Products)		
Salable Products (Parts)	300K - 1M	• Saleable Products have Stock records	Commerce Platform (Products)		
Inventory (Stock)	300K	• Multiple parts (differing by brand name only) can be fulfilled by a single Stock item.	Commerce Platform (Product)		
Brands	30	• Parents, Models			

Brand	30*30	• Catalog Parent			
Product Families	?				
Equipment Types	“				

Products

Current Product schema

```

1  {
2    "type": "part",
3    "url": "kawasaki-11004-7008",
4    "title": "11004-7008",
5    "description": "Head Gasket",
6    "notes": null,
7    "stock": "CSYSKAW 11004-7018",
8    "keywords": "110047008 kawasaki gasket head 11004 7008 110047008 kawasaki",
9    "id": "8d0ba19b-c88a-5887-a753-bf6934c04a25",
10   "source": "ARIPS29PRT 11004-7008",
11   "meta": {
12     "inserted": "2023-10-01T05:00:00+00:00",
13     "updated": "2023-02-12T19:05:46+00:00",
14     "approved": true,
15     "deleted": false,
16     "comments": null,
17     "review": null
18   },
19   "photos": [
20     "CSYSKAW 110047018 566f38d512b67",
21     "CSYSKAW 110047018 566f38d6d4351"
22   ],
23   "supersedes": [
24     "ARIPS29PRT 11004-7018"
25   ],
26   "variants": [],
27   "bundles": [],
28   "manufacturing_brand": [
29     "Kawasaki"
30   ],
31   "marketing_brand": [
32     "Kawasaki"
33   ],
34   "equipment_type": [],
35   "searchable": [
36     true
37   ],
38   "velocity": [
39     11
40   ],
41   "long_description": [
42     ""
43   ],
44   "part_type": [
45     "Gasket"
46   ],
47   "part_name": [
48     "gasket",

```

```

49     "head"
50   ]
51 },
52 {
53   "type": "part",
54   "url": "toro-commercial-11004-7008",
55   "title": "11004-7008",
56   "description": "GASKET-HEAD",
57   "notes": null,
58   "stock": null,
59   "keywords": "110047008 toro 11004 7008 110047008 toro",
60   "id": "3e034b17-8269-5d59-8bda-8e62e3a4df1c",
61   "source": "ARIPS53PRT 11004-7008",
62   "meta": {
63     "inserted": "2023-08-01T05:00:00+00:00",
64     "updated": "2023-02-11T21:27:59+00:00",
65     "approved": true,
66     "deleted": false,
67     "comments": null,
68     "review": null
69   },
70   "part_number": [
71     "11004-7008"
72   ],
73   "manufacturing_brand": [
74     "Toro"
75   ],
76   "marketing_brand": [
77     "Toro Commercial"
78   ],
79   "searchable": [
80     true
81   ]
82 },

```

- can be added to Commerce Catalog
- ~~saleable subset?~~ Currently planing on including all
- custom attributes can be maintained
 - does indexing of certain attributes into a “dynamic category” fulfill some need for Model/IPL?
 - list of all model values via product→models attribute ?
 - list of all products with model=abc123 ?
 - part equivalency (multiple marketing brands per single product)

Models

- not a good fit for native categories
 - volume (300K)
- product attribute for models
 - reference to id to retrieve data
- need to maintain external tables with Model records to display on site

- each with reference to IPLs / Products

IPLs/Parts Diagrams

- Could function as a category for products except
 - IPLs can be shared amongst multiple models
 - category custom attributes lacking in commerce platforms
- option 1: maintain external tables with IPL records to display on site
- option 2: ✅ IPLs as Categories
 - approved by Shopware
 - separate category tree for IPLS
 - linked to multiple Models via Custom Field - Entity Relationship type

Platform Implications

Possibilities for Extending Data Model?

- Product Attributes
- Category Attributes
- Custom Entities

Features that reduce overall Dev effort?

- extending
- native

Feature/Benefit of platform for building on vs custom legacy approach?

Shopware

Custom Entities

<https://developer.shopware.com/docs/guides/plugins/overview.html> Extension comparison

Apps (SaaS+PaaS) Vs Plugins (PaaS only)

Custom Entities are possible (IPLs, Models?)

The PartsTree approach would require not breaking the easy upgrade path for shopware versions.

from Vance:

Breaking:

- Making changes to an existing CMS Block
- Altering an existing entity field

Non-breaking

- Creating a new CMS block
- Creating a new entity

•

Sub-System	Catalog Ingestion and Sync
Team	@Sam Bogart
Status	Not Started

Headless Storefront

[Summary](#)

[Decisions](#)

[Tech Stack](#)

[Architecture](#)

[MVP](#)

[Platform Implications](#)

[Shopware](#)

Summary

Current: Fully Custom Ecommerce built in PHP/MVC + AWS DocumentDB

MVP: Replace with Shopware + Headless Storefront (Shopware Frontends) [Shopware Documentation](#)

Future State: Continue adding functionality, potential future 3P CMS

Decisions

Question	Answer	Reasoning	Date
Which frontend framework?	Use Shopware Frontends(Vue/Nuxt) Shopware Frontends Internal Structure	Pre-built functionality that is extendable, customizable, overwritable as needed supports us to build efficiently, using Shopware endorsed and developed tooling. Vue/Nuxt also is the Admin FE framework, so customizations to it would also use	
CMS?	Relying on Shopware native CMS (aka Shopping Experiences) for MVP	OOTB functionality should be sufficient	

Where to host headless storefront?	Need to discuss with Webscale (our Shopware hosting provider) if that is included or if extra does it make sense to keep there vs Cloudflare/Vercel/etc.		
SW built-in Elasticsearch sufficient?			

Tech Stack

driven by Frontends: [Overview](#) | [Shopware Frontends](#)

Commerce Platform: Shopware 6.7

Server: Node.js

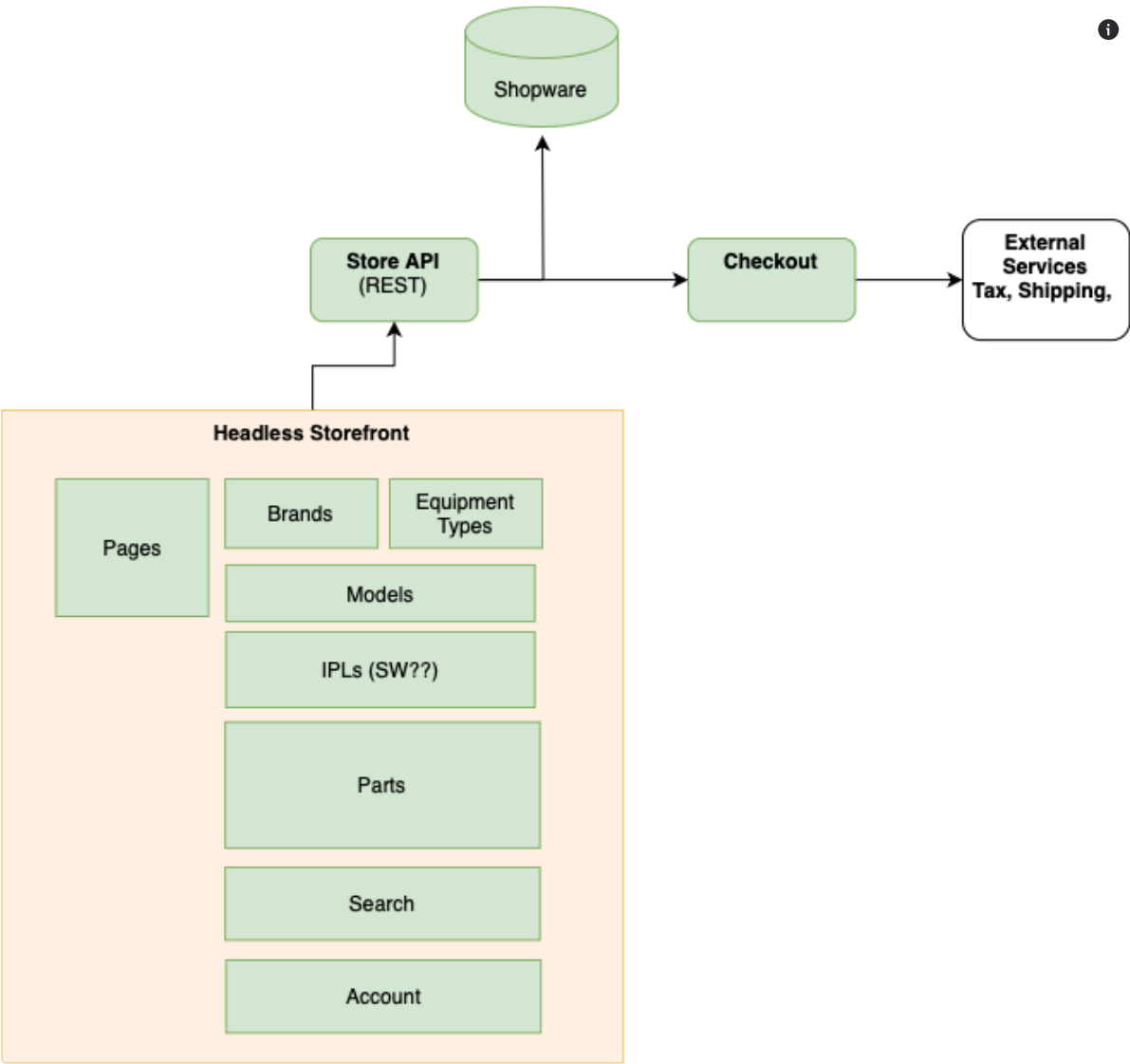
FE:

- Vue 3 [Vue.js](#)
- Nuxt 3 [Nuxt: The Full-Stack Vue Framework](#)
- Typescript

CSS: Unocss: [UnoCSS](#)

Architecture

MVP



Platform Implications

Shopware

-

Sub-System	Storefront
Team	@Sam Bogart
Status	Not Started

Storefront Model & Part Page Data Requirements

Model Page Data:

- Model Info
- IPLs
 - Image, Hotspots, Products
- Model Products

All from Shopware

Initial Page Load (pre-interaction)

Get Model Info

- SW Store API/Composable: /store-api/category/[id] (useCategorySearch composable)
 - includes IPL category list (customFields. (ids)
- Extend or Custom API route to retrieve IPLs (custom field entity selection) with Model info?
 - including (ids, name, image url)

Get Model Products (first 100 limit)

- SW Store API/Composable: /store-api/product-listing/[id] (useListing composable)

Get All IPL base data (If need separate call)

- SW Store API/Composable: /store-api/category/ with { ids: [], includes: {} } in request,
 - TBD is media image is on SW or kept in existing s3 with url as custom field?

```
1 {
2   "associations": {
3     "media": {}
4   },
5   "includes": {
6     "category": [
7       "id",
8       "name",
9       "media",
10      "seoUrls",
11      "customFields"
12    ],
13     "media": [
14       "id",
15       "url",
16       "title"
17     ]
18   },
19   "ids": [
20     "019672150d4471618bdca86a0e021805",
```

```
21     "019669552e2577e7b2e2fe532b3d2fc2"  
22   ]  
23 }
```

On Demand IPL Products Filter

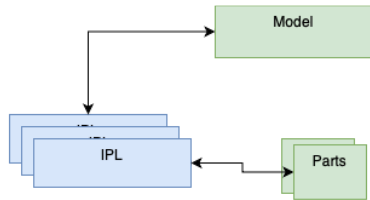
Get IPL Detail (products, hotspots, image)

- SW Store API/Composable: /store-api/category/[id] (useCategorySearch composable)
 - with hotspots as custom fields (JSON string added directly to category not easily managed in Admin UI)
 - custom fields (hotspots) not included in default product-listing response
 - IPL reference # (defined by IPL graphic)
 - add SW product ref (new property needs to be added to hotspots[] object)
 - IPL image - continue to use Cloudfront image hosting as is?

ALT IPLs from DB

backup if IPLs as Categories in SW doesn't work out.

- get IPLs from Model ID
 - IPLs[]
 - products, image, hotspots

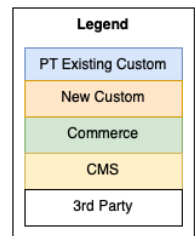
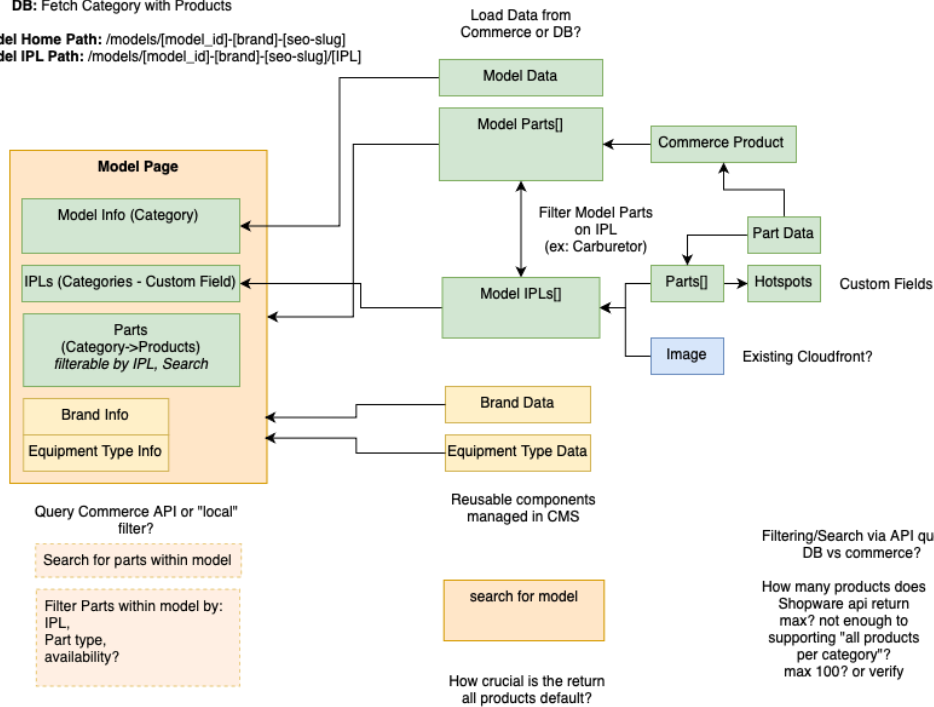


Commerce: Fetch Category with Products

CMS: Fetch CMS Page Data?

DB: Fetch Category with Products

Model Home Path: /models/{model_id}-{brand}-{seo-slug}
Model IPL Path: /models/{model_id}-{brand}-{seo-slug}/{IPL}



Brand Equipment Type Page

brands/[toro]/[walk-behind-mowers]/

SW CMS:

- Header Image
- Related Content Carousel Cards

Category Listing:

- Brand Product Families
 - links to other families/types for brand
- Product Family Models List
 - Models
 - Model Variants
 - Image

Brand Home

CMS Landing Page

Header Image

Intro content text

Product Family/Equipment Type list

- Product Families are similar to Equipment Types (but sometimes have the Brand's trademarked naming convention (Snow Thrower, etc)
- Current PT doesn't track these as own entities, they are aggregated from Parts properties per brand -
 - Products Table - type: Models have data `product_family` and `equipment_type` , product families for listing and equipment type as a filter, property for models?
 - load as Categories in Shop-ware
 - full tree ? Echo → Chainsaws → CS-100 or Echo Chainsaws → CS-100
 -

Recently Viewed Models

Default Components

- Need Help?
- Shipped Fast!
- Popular Products Carousel

Product Page

<https://www.partstree.com/parts/troy-bilt-45-883-02-s1>

/parts/[brand]-[number]

TBD base on design

Product Images

Product Info

Catalog and Legacy Data Ingestion & Sync for MVP

Summary

Current State

Proposal for MVP

TBD / Decisions

Sub-System Architecture

MVP

Migration Possibilities (WIP)

Models → IPLs, IPLs → Parts

ETL Steps Overview:

Products

Models

IPLs/Parts Diagrams

Platform Implications

Shopware

Summary

Current State

Cataloger (UI, Batch Catalog Manipulation), **Catalog DB** (Local MongoDB) to [PartsTree.com](https://partstree.com) (AWS DocumentDB)

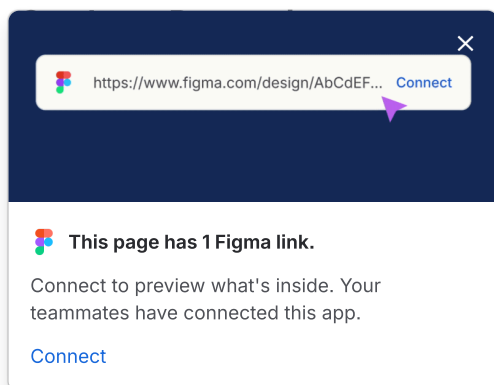
The first version of the cataloger was local, and developers would modify data offline and then upload it to the on-premises catalog when they returned

The current process involves several key steps:

Data Harvesting:

Data sources used to come physically on CD, and the amount of data was significant enough that they preferred to keep it on-premises. The data transfer would be easier locally

They still receive CDs for most brands, and the process involves ripping data from the CD on a local machine to process it into the catalog.



ent flags in its system:

ment is scheduled for sync

one importing data can mark something for review

s someone has QA'd the data and deemed it correct for the

Data Preparation and Sync:

The process is managed by a CRON job with a scheduled time known to everyone. When the ETL process kicks off:

- It uses metadata filters
- Checks if items are approved
- Checks if items are deleted
- Sends appropriate data to the website DB

Website Integration:

The catalog interfaces with the website through the website's API. The website doesn't have direct knowledge of the cataloger. Everything the catalog does is to send data to the website's API

DocumentDB and Elasticsearch:

- Models don't directly go into DocumentDB like products, but rather go into DocumentDB primarily for the purpose of indexing into Elasticsearch

The purpose of these models is purely informational - for search and fast reading. There's no frequent need to update individual pieces of data.

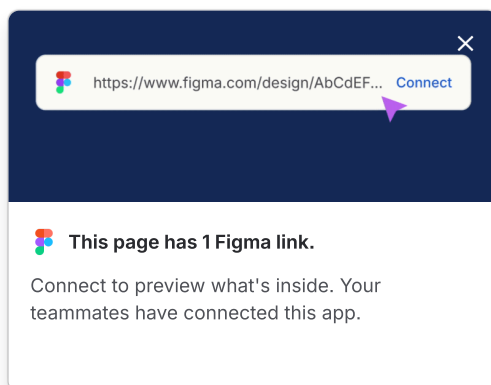
Additional Nuances:

The cataloger is intentionally designed to be agnostic of the website. It was built so that any additional services can consume its data. The cataloger doesn't care that it's sending data to the website - it's meant to be a flexible data source

Proposal for MVP

Database Location and Access:

The proposal is to move the master catalog (currently a MongoDB) into the AWS VPC for



also involve moving the cataloger software that manages the

the cloud that:

manage

for Shopware loads and syncs

- Maintains important properties that dictate whether items are ready to update in Shower

Preservation of Existing Processes:

The approved flag would remain a top-level indicator of whether data is correct and ready for other services to use. Any ingestion service would need to respect this existing approval mechanism.

Approval and Sync Mechanism:

proposing a system with:

- A queue of changes
- Visibility into what changes are queued
- A way to show old vs. new data (like code diffs, but more user-friendly)
- A manual user approval process for safety and execution of changes to Shopware

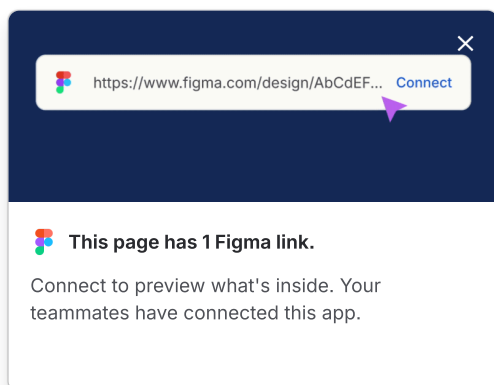
Future Evolution:

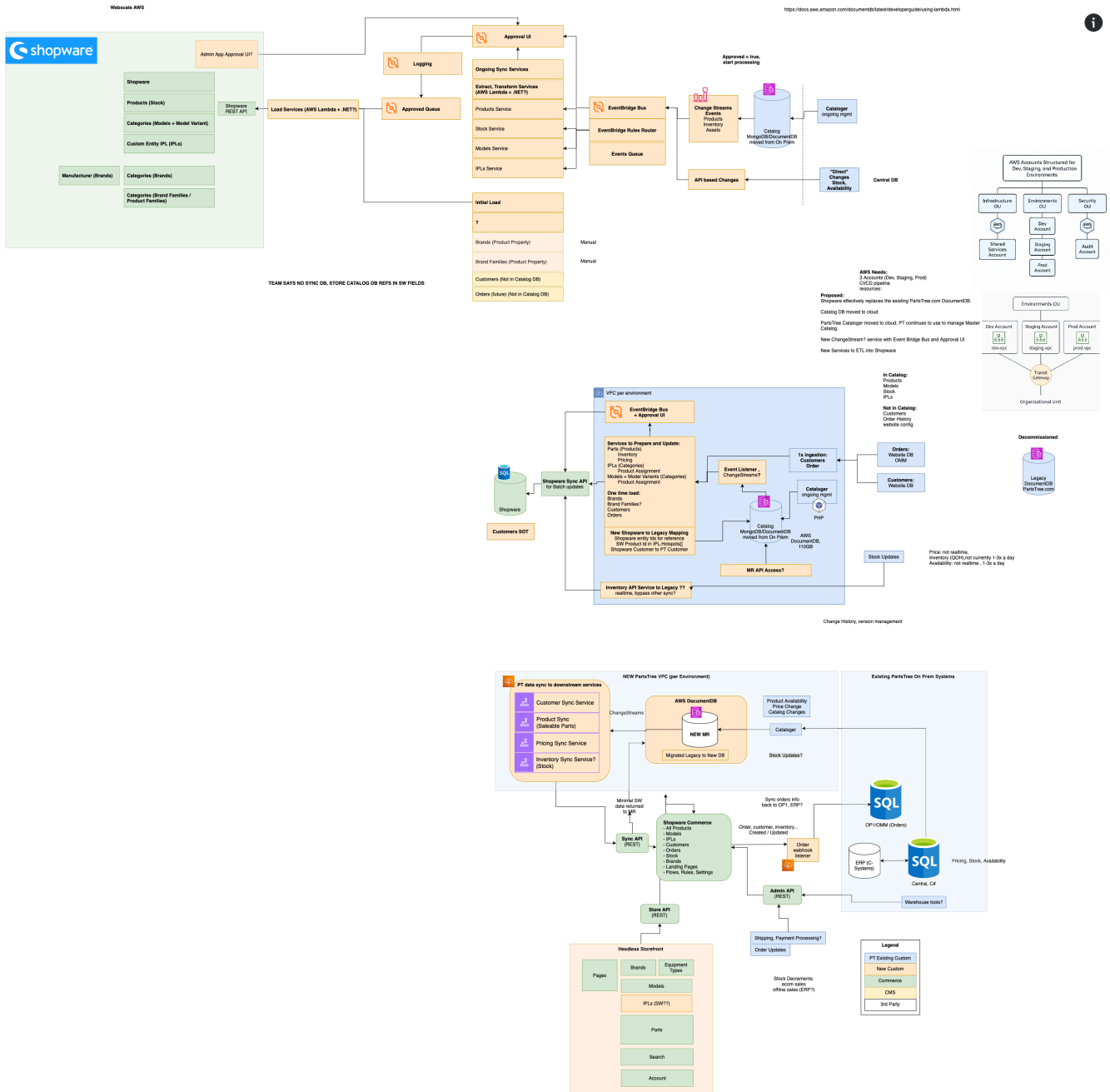
Eventually, they might:

- Leverage a PIM (Product Information Management) to replace cataloger functionality
- Rebuild cataloger functionality inside the PIM
- Maintain the core system of services needed for this evolution

Key Principles:

The cataloger was originally built to be service-agnostic. It doesn't care about the website and was specifically designed so that any additional services can consume its data. Altus Nova coming in and using the cataloger is exactly what it was built for.





<https://www.figma.com/design/AbCdEF...>
Connect

This page has 1 Figma link.

Connect to preview what's inside. Your teammates have connected this app.

all Product Models, IPLs, Equipment Types as

for new storefront Catalog and Page

Reasoning	Date

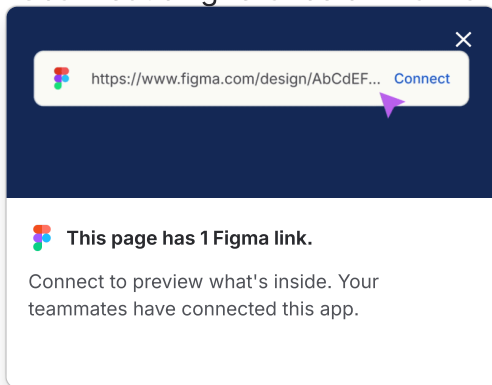
system?	content to come from Shopware Store API	Categories, Parts as Products.	
Syncing Mechanism	Catalog DB → EventBridge?		
Only Saleable Parts into Commerce?	No, All Products	Expecting all Products, Saleable or Not to live in Shopware	
Models in Commerce ?	Yes, as Categories	Shopware approved	
IPLs in Commerce?	Yes, as Categories	Shopware approved	
Supersedes	Custom Field? (Entity Reference)		
Future state cataloger	TBD Post MVP		
Products to Models Mapping			

Sub-System Architecture

MVP

Migration Possibilities (WIP)

Goal: reducing reliance on maintaining current catalog db



part equivalency transparency for customer

“recommended products” , but has connected IPL

ch combine to hold full list of Model Products (combined

IPL subsets)

- less sync processes to populate parts in both Category sets
- cleaner?
- **Option 2: Model has Parts mapped and IPLs have Parts mapped.**
 - extra effort in sync process to populate all the values twice?
 - updates to a part mapping to IPL need to also update Model in SW
 - more possibilities for data retrieval

Use Cases:

1. Parts without IPLs (%?)
2. Search within a Model:  <https://www.figma.com/design/W2QS70KmuK3VQkABdFERLv/Part sTree---Drafts?node-id=1309-11296&t=StKVZTCdTUT0Yuog-1> [Connect your Figma account](#)
 - a. text search to filter list of parts
 - b. need to filter by type as well (belts, bolts, etc)
3. Show and Search all of a Model's Parts
 - a. ability to scroll through all (cumbersome if many hundreds of parts per model)
4. Render list of Models that a Part fits
 - a. API call to get product 's IPLs?
 - i. 1 POST {url}/store-api/product/{productId}?associations[0]=categories&filter[category.tagIds][0]={tagId}
 - b. Product → Categories (filtered by Tag, or Parent, custom field?)
 - i. /store-api/product/{productId} with body like:

```
1 {
2   "associations": {
3     "categories": {
4       "associations": {
5         "tags": {}
6       }
7     }
8   },
9   "includes": {
10    "product": ["id", "name", "categories"],
11    "categories": ["id", "name", "tags"],
12    "tags": ["id", "name"]
13  }
```

 <https://www.figma.com/design/AbCdEF...> [Connect](#)

 This page has 1 Figma link.

Connect to preview what's inside. Your teammates have connected this app.

Current Catalog Schemas:

Assets Collection: IPL.parts, IPL.hotspots has Part/Product references

```
1  "parts": [  
2    {  
3      "reference": "1",  
4      "quantity": null,  
5      "notes": [  
6        "Included in Air cleaner ass'y"  
7      ],  
8      "title": "13031300132",  
9      "description": "COVER, AIR FILTER",  
10     "product": "ARIPS61PRT 13031300132"  
11   },  
12   ...  
13 ],  
14 "hotspots": [  
15   {  
16     "reference": "1",  
17     "x": 52.295918367347,  
18     "y": 2.0186335403727,  
19     "notes": null,  
20     "author": null  
21   },  
22   ...  
23 ]
```

Products Collection: Model.ipl has array of IPL references

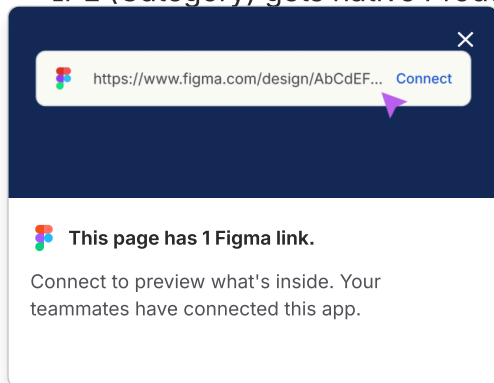
```
1  "ipl": [  
2    "ARIPS61IPL 01006970003",  
3    "ARIPS61IPL 01006970007",  
4    ...  
5  ],
```

Products Collection: Part.source has IPL reference:

```
1  "source": "ARIPS61PRT 90060500005"
```

MVP:

- IPL (Category) gets native Products mapping to SW Products



exist in PT catalog)

s mapping to SW Products

/ exist in PT catalog)

doesn't currently exist in PT catalog)

- Prerequisites:

- add SW ID to PT catalog or add SW ID to a Sync DB
- need to now SW Product IDs before mapping IPL, Model to Products

ETL Steps Overview:

Possible approaches:

Load “container” entities before mapping, then update entities with mapping, detailed data??

1. Load all brands (marketing and manufacturing?) into SW as Manufacturers
 - a. return SW Ids to Sync DB (write the brands to sync db 1st?) ????
2. Load all brand families as Categories?
3. Load all parts into SW as Products
 - a. return SW Ids to Sync DB (write the products to sync db 1st?)
4. Load all models into SW as Categories (differentiating properties, tags for type?)
 - a. return SW Ids to Sync DB (write the models to sync db 1st?)
5. Load all IPLs into SW as Categories (differentiating properties, tags for type?)
 - a. return SW Ids to Sync DB (write the IPLs to sync db 1st?)
 - b.

Name	Volume	Catalog Relationship	New Location	Search Impact	Management Impact
Models	300K	Models connect to 10s of IPLs	PartsTree DB? Commerce Platform (Categories)	External Search Req if not in Commerce	


<https://www.figma.com/design/AbCdEF...>
Connect



This page has 1 Figma link.

Connect to preview what's inside. Your teammates have connected this app.

		are connected to Models			
		•			
Model Parent		• Model Parent can have many “Model Variants”	PartsTree DB? Commerce Platform (Categories)		
IPLs / Parts Diagrams	1M	• IPLs can connect to many Models • IPLs connect to 10s of Parts	PartsTree DB? Commerce Platform (Categories)		
Total Products (Parts)	3.5M	• Not currently connected to Models • List of Models not currently an attribute • List of IPLs not currently an attribute	Commerce Platform (Products)		
Salable Products (Parts)	300K - 1M	• Saleable Products have Stock records	Commerce Platform (Products)		
Inventory (Stock)	300K	• Multiple parts (differing by brand name only) can be fulfilled by a single Stock item.	Commerce Platform (Product)		
Brands	30	• Parents, Models			
		g Parent			


<https://www.figma.com/design/AbCdEF...>
Connect



This page has 1 Figma link.

Connect to preview what's inside. Your teammates have connected this app.

Products

Current Product schema

```
1  {
2    "type": "part",
3    "url": "kawasaki-11004-7008",
4    "title": "11004-7008",
5    "description": "Head Gasket",
6    "notes": null,
7    "stock": "CSYSKAW 11004-7018",
8    "keywords": "110047008 kawasaki gasket head 11004 7008 110047008 kawasaki",
9    "id": "8d0ba19b-c88a-5887-a753-bf6934c04a25",
10   "source": "ARIPS29PRT 11004-7008",
11   "meta": {
12     "inserted": "2023-10-01T05:00:00+00:00",
13     "updated": "2023-02-12T19:05:46+00:00",
14     "approved": true,
15     "deleted": false,
16     "comments": null,
17     "review": null
18   },
19   "photos": [
20     "CSYSKAW 110047018 566f38d512b67",
21     "CSYSKAW 110047018 566f38d6d4351"
22   ],
23   "supersedes": [
24     "ARIPS29PRT 11004-7018"
25   ],
26   "variants": [],
27   "bundles": [],
28   "manufacturing_brand": [
29     "Kawasaki"
30   ],
31   "marketing_brand": [
32     "Kawasaki"
33   ],
34   "equipment_type": [],
35   "searchable": [
36     true
37   ],
38   "velocity": [
39     11
40   ],
41   "long_description": [
42     ""
43   ],
44   "part_type": [
45     "Gasket"
46   ],
```



<https://www.figma.com/design/AbCdEF...> [Connect](#)



This page has 1 Figma link.

Connect to preview what's inside. Your teammates have connected this app.

11004-7008",

HEAD",

oro 11004 7008 110047008 toro",

60 "id": "3e034b17-8269-5d59-8bda-8e62e3a4df1c",

```

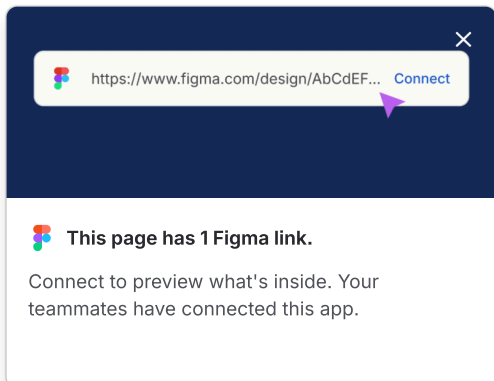
61     "source": "ARIPS53PRT 11004-7008",
62     "meta": {
63         "inserted": "2023-08-01T05:00:00+00:00",
64         "updated": "2023-02-11T21:27:59+00:00",
65         "approved": true,
66         "deleted": false,
67         "comments": null,
68         "review": null
69     },
70     "part_number": [
71         "11004-7008"
72     ],
73     "manufacturing_brand": [
74         "Toro"
75     ],
76     "marketing_brand": [
77         "Toro Commercial"
78     ],
79     "searchable": [
80         true
81     ]
82 },

```

- can be added to Commerce Catalog
- salable subset? Currently planing on including all
- custom attributes can be maintained
 - does indexing of certain attributes into a “dynamic category” fulfill some need for Model/IPL?
 - list of all model values via product→models attribute ?
 - list of all products with model=abc123 ?
 - part equivalency (multiple marketing brands per single product)

Models

- not a good fit for native categories
- product attribute for models
 - reference to id to retrieve data
- need to maintain external tables with Model records to display on site



Products

products except

multiple models

working in commerce platforms

- likely need to maintain external tables with IPL records to display on site

Platform Implications

Possibilities for Extending Data Model?

- Product Attributes
- Category Attributes
- Custom Entities

Features that reduce overall Dev effort?

- extending
- native

Feature/Benefit of platform for building on vs custom legacy approach?

Shopware

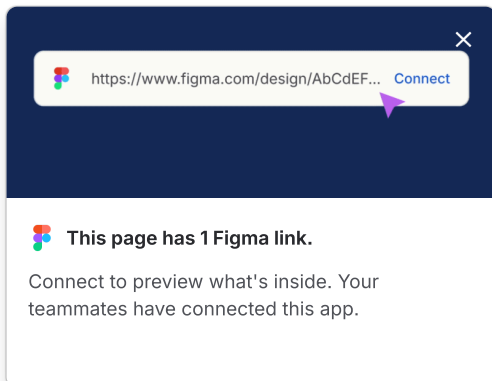
Custom Entities

<https://developer.shopware.com/docs/guides/plugins/overview.html> Extension comparison

Apps (SaaS+PaaS) Vs Plugins (PaaS only)

Custom Entities are possible (IPLs, Models?)

[Custom Entities | Shopware Documentation](#)



require not breaking the easy upgrade path for shopware

from Vance:

Breaking:

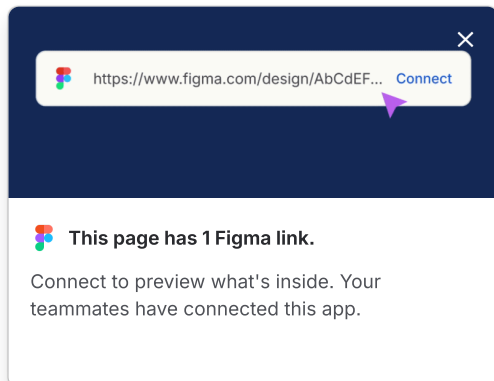
- Making changes to an existing CMS Block
- Altering an existing entity field

Non-breaking

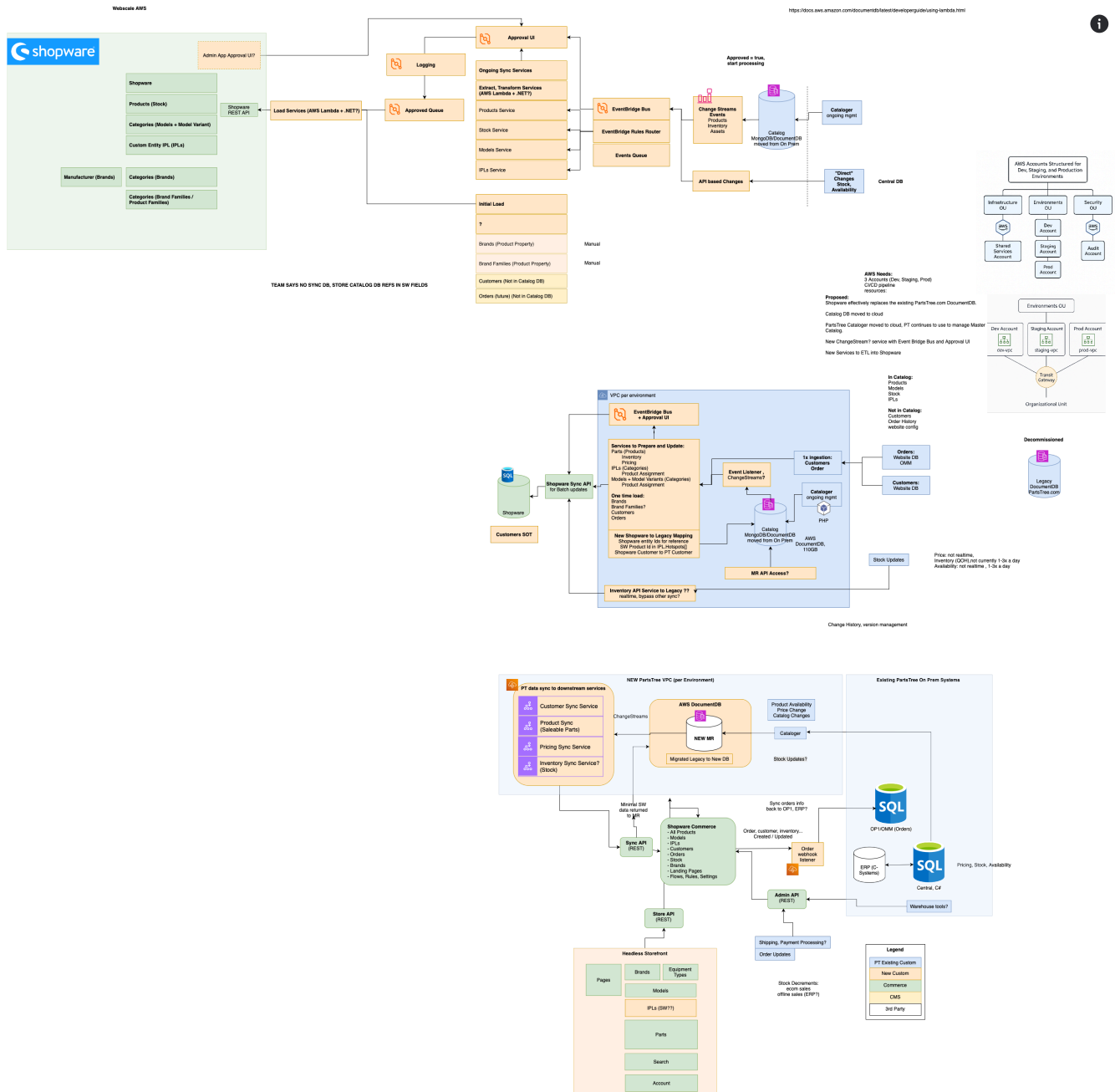
- Creating a new CMS block
- Creating a new entity

•

Sub-System	Catalog Ingestion and Sync
Team	@Sam Bogart
Status	Not Started



BE TEAM TECH DESIGNS



Tech Design: Change Stream Consumer

Overview

Goal - launch the service that handles change stream events from DocumentDB and publishes events to Event Bridge.

The service should be deployed to AWS ECS.

List of tasks

#	Task	Description	Estimation, h	Jira	Notes
1	Setup repo	Create a new repo for a .NET app. Acceptance criteria: • The “change-stream-consumer” repo is created • The <i>master</i> and <i>development</i> branches are created • The <i>master</i> branch is protected • The <i>.gitignore</i> is added	2h	PAR-194: Setup repo DONE	
2	Setup project	Create a skeleton for a .NET console app. Acceptance criteria: • Structured logging added (Serilog, CloudWatch) • AWS SSM is integrated • Connection credentials for DocumentDB and Event Bridge are configurable • A Docker file is created, and a Docker image is built • Service is deployed to ECS cluster manually	4h	PAR-200: Set up project DONE	
3	Implement	Create a module that activates Change Streaming at Catalog DB, connects to the stream, and	16h	PAR-196: I	Check Event

	consumer	pushes received events to Event Bridge Assumption: the received event should be pushed to Event Bridge as it is, no processing needed Acceptance criteria: when update/insert/delete happens in Catalog DB, an event should be visible in Event Bridge		mplement consumer r DONE	Bridge limitations for event size
4	Set up CI/CD	Set up a CI/CD pipeline for Change Stream Consumer repository. Acceptance criteria: - static analysis is run, - unit and integration tests is run, - Docker image is built, - Docker image is pushed to ECR, - ECS service is updated	16h	PAR-195: Setup CI/CD pipeline DONE	

Tech Design: EventBridge Bus Service

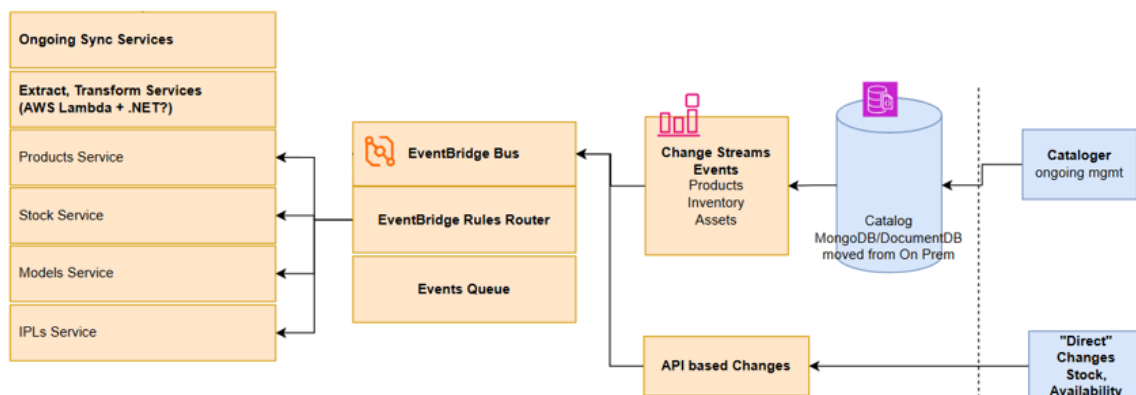
Goal – Set up and launch the infrastructure and routing logic of the EventBridge Bus, which orchestrates the flow of sync-ready catalog events from multiple sources - Change Stream Events or API-based changes - to downstream extract-transform services (Products, Stock, Models, IPLs).

The EventBridge Bus serves as the routing hub that:

- Subscribes to events generated from changes in the Catalog DB (DocumentDB) and from external API calls.
- Applies rules via the EventBridge Router to determine target services.
- Delivers filtered events to appropriate sync service processors built with AWS Lambda or .NET ECS containers.

Implementation includes:

- Creation of the EventBridge Bus
- Definition of routing rules per event type
- Permissions setup
- Integration with logging and auditing mechanisms



Product Event Schema

```
1 {  
2   "$schema": "http://json-schema.org/draft-07/schema#",  
3   "title": "CatalogSyncProductEvent",
```

```

4  "type": "object",
5  "properties": {
6    "version": { "type": "string" },
7    "id": { "type": "string", "format": "uuid" },
8    "detail-type": { "const": "catalog.sync" },
9    "source": { "const": "catalog.db" },
10   "account": { "type": "string" },
11   "time": { "type": "string", "format": "date-time" },
12   "region": { "type": "string" },
13   "resources": { "type": "array", "items": { "type": "string" } },
14   "detail": {
15     "type": "object",
16     "properties": {
17       "type": { "const": "product" },
18       "action": { "enum": ["create", "update", "delete"] },
19       "payload": {
20         "type": "object",
21         "properties": {
22           "id": { "type": "string" },
23           "sku": { "type": "string" },
24           "title": { "type": "string" },
25           "description": { "type": "string" },
26           "brand": { "type": "string" },
27           "manufacturingBrand": { "type": "string" },
28           "sourceIplId": { "type": "string" },
29           "stockQuantity": { "type": "number" },
30           "price": { "type": "number" },
31           "isSalable": { "type": "boolean" },
32           "updatedAt": { "type": "string", "format": "date-time" }
33         },
34         "required": ["id", "sku", "title", "brand", "updatedAt"]
35       }
36     },
37     "required": ["type", "action", "payload"]
38   }
39 },
40 "required": ["version", "id", "detail-type", "source", "time", "region", "detail"]
41 }

```

Model Event Schema

```

1  {
2    "$schema": "http://json-schema.org/draft-07/schema#",
3    "title": "CatalogSyncModelEvent",
4    "type": "object",
5    "properties": {
6      "version": { "type": "string" },
7      "id": { "type": "string", "format": "uuid" },
8      "detail-type": { "const": "catalog.sync" },
9      "source": { "const": "catalog.db" },
10     "account": { "type": "string" },
11     "time": { "type": "string", "format": "date-time" },
12     "region": { "type": "string" },
13     "resources": { "type": "array", "items": { "type": "string" } },
14     "detail": {
15       "type": "object",
16       "properties": {
17         "type": { "const": "model" },
18         "action": { "enum": ["create", "update", "delete"] },
19         "payload": {
20           "type": "object",
21           "properties": {
22             "id": { "type": "string" },

```

```

23     "name": { "type": "string" },
24     "brand": { "type": "string" },
25     "brandFamily": { "type": "string" },
26     "variant": { "type": "string" },
27     "iplRefs": {
28         "type": "array",
29         "items": { "type": "string" }
30     },
31     "equipmentType": { "type": "string" },
32     "isVariant": { "type": "boolean" },
33     "updatedAt": { "type": "string", "format": "date-time" }
34 },
35     "required": ["id", "name", "brand", "updatedAt"]
36 },
37 },
38     "required": ["type", "action", "payload"]
39 },
40 },
41     "required": ["version", "id", "detail-type", "source", "time", "region", "detail"]
42 }

```

IPL Event Schema

```

1  {
2    "$schema": "http://json-schema.org/draft-07/schema#",
3    "title": "CatalogSyncIPLEvent",
4    "type": "object",
5    "properties": {
6      "version": { "type": "string" },
7      "id": { "type": "string", "format": "uuid" },
8      "detail-type": { "const": "catalog.sync" },
9      "source": { "const": "catalog.db" },
10     "account": { "type": "string" },
11     "time": { "type": "string", "format": "date-time" },
12     "region": { "type": "string" },
13     "resources": { "type": "array", "items": { "type": "string" } },
14     "detail": {
15       "type": "object",
16       "properties": {
17         "type": { "const": "ipl" },
18         "action": { "enum": ["create", "update", "delete"] },
19         "payload": {
20           "type": "object",
21           "properties": {
22             "id": { "type": "string" },
23             "title": { "type": "string" },
24             "description": { "type": "string" },
25             "imageUrl": { "type": "string", "format": "uri" },
26             "author": { "type": "string" },
27             "linkedModelIds": {
28               "type": "array",
29               "items": { "type": "string" }
30             },
31             "hotspots": {
32               "type": "array",
33               "items": {
34                 "type": "object",
35                 "properties": {
36                   "reference": { "type": "string" },
37                   "x": { "type": "number" },
38                   "y": { "type": "number" },
39                   "productSku": { "type": "string" },
40                   "quantity": { "type": "number" },

```

```

41         "notes": { "type": "string" }
42     },
43     "required": ["reference", "x", "y"]
44 },
45 },
46     "updatedAt": { "type": "string", "format": "date-time" }
47 },
48     "required": ["id", "title", "updatedAt"]
49 },
50 },
51     "required": ["type", "action", "payload"]
52 },
53 },
54     "required": ["version", "id", "detail-type", "source", "time", "region", "detail"]
55 }

```

List of tasks

#	Task	Description	Estimation, h	Jira	Notes	
1	Define and create EventBridge Bus	Provision a custom EventBridge Bus via Terragrunt Acceptance criteria: • Bus is provisioned and visible in AWS Console • Bus name follows project naming convention • Accessible from dev account	8h	🔗 PAR-20 1: Define a nd create EventBridge Bus DONE		
2	Define event structure	Define standard event schema for catalog sync events Acceptance criteria: • JSON schema is documented and reviewed • Sample events are shared and validated • Supported types: Product, Model, IPL	4h	🔗 PAR-20 2: Define event structure DONE		
3	Implement routing rules	Create routing rules for Products, Stock, Models, and IPLs	6h	🔗 PAR-20 3: Implement		

		Acceptance criteria: • Routes to appropriate sync target • Events are matched correctly in test		ent routing rules DONE		
4	Configure IAM permissions	Set up IAM policies and principals for allowed event publishers and consumers Acceptance criteria: • Only allowed producers can put events • Only target services can consume	4h	PAR-20 4: Configure IAM permissions DONE		
5	Integrate with logging	Enable CloudWatch logging and auditing for event delivery, filtering, and failures Acceptance criteria: • Event logs are visible in CloudWatch • Optional alarms configured	4h	PAR-20 5: Integrate with logging DONE		
6	Create DLQ for failed events	Add SQS DLQ to capture undelivered/failed events for diagnostics Acceptance criteria: • DLQ is provisioned and attached to ApprovedQueue • Delivery attempts and failure threshold are configured • Messages move to DLQ after N retries • DLQ visible in AWS Console and accessible by diagnostics team	4h	PAR-20 5: Integrate with logging DONE	PA R-2 06: Create DLQ for failed events DONE	

Tech Design: IPL Custom Entity Plugin

Overview

Goal – Implement IPLs (Illustrated Parts Lists) as a **Custom Entity** in Shopware via a plugin.

The entity supports many-to-many associations with:

- **Products** (`product`)
- **Models** (`category`)

This allows IPLs to be:

- Fully integrated with Shopware APIs
- Stored in a dedicated table with relationships
- Synced from the legacy Catalog DB

Scope

- Define `ipl` Custom Entity with relationships
- Add join tables for `ipl_product` and `ipl_model`
- Sync data from CatalogDB via EventBridge-triggered AWS Lambdas
- Create “mock” entities when dependencies are missing
- Provide visibility control using `visible` flag
- Support `/api/_action/sync` for IPL creation or upserts

Data Model

IPL Entity

Field	Type	Source
<code>id</code>	UUID	Generated
<code>catalogId</code>	string	<code>asset.id</code>
<code>title</code>	string	<code>asset.ipl.title</code>
<code>description</code>	string	<code>asset.ipl.description</code>
<code>hotspots</code>	JSON	from <code>hotspots + parts</code>

<code>media_id</code>	FK to Media	s3 hosted image
<code>products</code>	ManyToMany	<code>product</code> entity
<code>models</code>	ManyToMany	<code>category</code> entity
<code>created_at</code>	datetime	auto

Join Tables

IPL_model

Field	Type
<code>ipl_id</code>	UUID
<code>model_id</code>	UUID
<code>created_at</code>	datetime

IPL_product

Field	Type
<code>ipl_id</code>	UUID
<code>product_id</code>	UUID
<code>created_at</code>	datetime

Data Loading Flow

1. IPL sync is triggered via **AWS EventBridge**, reacting to events from the **CatalogDB (DocumentDB)**.
2. Events may represent **insert**, **update**, or **delete**.
3. IPL sync is handled by a **dedicated .NET AWS Lambda function**.
4. If referenced models or products do not yet exist:
 - IPL Lambda creates **mock** dependencies with `is_mock = true` and `visible = false`.
 - Mock dependencies are later **replaced** by real synced data.

Sync Payload Example

```

1  {
2    "write-ipl": {
3      "entity": "ipl",
4      "action": "upsert",
5      "payload": [
6        {
7          "id": "UUID",
8          "catalogId": "ARIPS61IPL 01006970003",
9          "title": "Engine Assembly",
10         "description": "Exploded view of engine",
11         "hotspots": [
12           {
13             "reference": "1",
14             "x": 52.29,
15             "y": 2.01,
16             "productId": "prod-uuid"
17           }
18         ],
19         "products": [{ "id": "prod-uuid" }],
20         "models": [{ "id": "cat-uuid" }]
21       ]
22     }
23   }
24 }

```

List of tasks

#	Task	Description	Estima tion, h	Jira	Note s
1	Create plugin base	Plugin skeleton & config Acceptance criteria: - custom/plugins/PartsTreeIplPlu gin directory is created - Plugin is registered and activated via bin/console plugin:refresh && plugin:install - Contains valid composer.json , manifest.xml , and plugin base class	4h	🔗 PAR-254: Create plug in base DONE	
2	Define IPL entity	Define IPL entity with all fields, and migration Acceptance criteria: ipl entity is defined in IplDefinition.php	4h	🔗 PAR-255: Define IPL entity DONE	

		- Required fields: <code>id</code>, <code>catalogId</code>, <code>title</code>, <code>description</code>, <code>hotspots</code>, <code>visible</code>, <code>is_mock</code> - Doctrine migration installs table to DB successfully			
3	Define join tables	Define <code>ipl_product</code> and <code>ipl_model</code> tables Acceptance criteria: <code>ipl_product</code> and <code>ipl_model</code> tables created and registered - Entities resolve associations via Admin API - Foreign keys exist and are enforced	2h	🔗 PAR-256: Define join tables DONE	
4	Admin API support	CRUD via Admin API Acceptance criteria: - IPLs are accessible via <code>/api/search/ipl</code> - Create, update, delete operations work via Admin API - Associations (products, models) are included in reads	4h	🔗 PAR-257: Admin API support DONE	
5	Mock resolution logic	Set up IAM permissions for producer to publish and sync processor to consume Acceptance criteria: - If real model/product arrives later, mock is replaced - Replaced IPL becomes <code>visible = true</code> - IPL record is updated with new references	8h	🔗 PAR-258: Mock resolution logic DONE	
6	Final validation	Link checks, test coverage	2h	🔗 PAR-259: Final validation DONE	

	Acceptance criteria:			
	• Sync tested with insert/update/delete IPL events • IPLs linked to correct Shopware entities • Hotspots JSON loads correctly into custom field • Admin API queries show correct associations			

Tech Design: Lambdas for ongoing sync service

Overview

Goal - implement synchronization between the CatalogDB data model and the Shopware data model:

- Model → Brand, Brand Family, Model. Model Variant(all as Category),
- Part → Manufacturer(built-in SW entity), Product(built-in SW entity), Product-To-IPL(service custom entity to allow many-to-many)
- IPL → IPL(Custom Entity), IPL-To-Model(service custom entity to allow many-to-many),

More details about mapping here: [☞ Sync approach - Short Breakdown by entities](#)

General approach:

1. Lambdas receive events about records changed in CatalogDB from Event Bridge.
2. There are 3 types of events: insert/update/delete.
3. An event can either relate to one particular entity(e.g., Model/Part/IPL update - description changed, Parent Model insert) or be dependent on another entity (e.g., Model insert, Model update - variant added).
4. If the entity related to the event is dependent on another entity, there are 2 possible situations:
 - a. The dependency exists in Shopware - the system just references it
 - b. The dependency does not exist in Shopware:
 - i. The System checks this dependency in the Catalog DB and, if it exists, inserts it into the Shopware
 - 🟡 The System can later receive an event that this dependency has been inserted - this event will be ignored
 - ii. If the dependency does not exist in the Catalog DB(e.g., a dependent entity was inserted earlier than a dependency), the system inserts a **“mock” dependency** that will be replaced with a real entity later
5. The System adds a special field to mark an entity as a “mock”.
6. Before inserting any new entity, the System checks whether there is a suitable “mock” to replace.
7. The System inserts new entities with the flag `visible = false`.
8. ! The **Catalog Tree** in Shopware can contain inconsistent sub-trees(with “mock” dependencies). These sub-trees are not visible at the Storefront.

9. A sub-tree of the Catalog Tree can be in one of the following states:
- Inconsistent - contains “mock” dependencies, not visible at the Storefront,  visible at Approval UI special area
 - Not Approved - does not contain “mock” dependencies, not visible at Storefront, visible at Approval UI and can be approved
 - Approved - does not contain “mock” dependencies, visible at Storefront, not visible at Approval UI
10. Approval UI loads data directly from SW API.

List of tasks

#	Task	Description	Estimation, h	Jira	Notes
1.	Setup repo	Create a new repo for a set of .NET Lambda projects. Acceptance criteria: - The “catalog-sync-functions” repo is created - The <i>master</i> and <i>development</i> branches are created - The <i>master</i> branch is protected - The <i>.gitignore</i> is added	2h	🔗 PAR-226: Setup repo 	
2.	Create a Terraform module	Create a module “catalog-sync-lambda” that consists of the following resources: <code>aws_lambda_function</code> - main resource defines Lambda <code>aws_lambda_permission</code> - permission for Event Bridge to trigger Lambda Acceptance criteria: - Pull request with the module is merged into <i>main</i> branch	4h	🔗 PAR-227: Create a Terraform module 	
3.	Create a Terragrunt unit	Create a unit from the module created at p2.	4h	🔗 PAR-228: Create a Terragrunt unit fo	

	for every lambda (3 units in total)	Acceptance criteria: The unit is deployed manually on dev environment		r every lambda da DONE	
4.	Setup a project for every lambda (3 projects in total)	Create a .NET AWS Lambda Function project. Use project templates: Building Lambda functions with C# - AWS Lambda Acceptance criteria: - Structured logging added (Serilog, CloudWatch) - AWS Secrets Manager is integrated - Lambda is deployed manually	4h	PAR-229: Setup a project for every lambda DONE	
5.	Setup CI/CD	Set up a CI/CD pipeline for the repo created at p1. Acceptance criteria: - for specific lambda if it has been changed: - static analysis is run, - unit and integration tests is run, - lambda built and deployed on dev	16	PAR-230: Setup CI/CD DONE	
6.	Implement Model sync	A Model MUST be mapped to SW as one of the following Categories: <i>Model</i> <i>Model Variant</i> Acceptance criteria: - A Model/Model Variant is created/updated/deleted “mock“ dependencies created if needed - Brand and/or Brand Family are created if not exist	32	PAR-231: Implement Model sync DONE	

		◦ never be “mock“			
7.	Implement Part sync	A Part MUST be mapped to SW Product. A manufacturing_brand property MUST be reflected to SW Manufacturer. Acceptance criteria: • A Part is created/updated/deleted ◦ “mock” dependencies • A Manufacturer is created, if not exist ◦ never be “mock“	16	PAR-232: I mplement Pa rt sync DONE	
8.	Implement IPL sync	An IPL MUST be mapped to SW Custom Entity. Acceptance criteria: • An IPL is created/updated/deleted ◦ “mock“ dependencies created if needed	24	PAR-233: I mplement IP L sync DONE	

🚫 [Eliminated] Tech Design: SQS

Overview

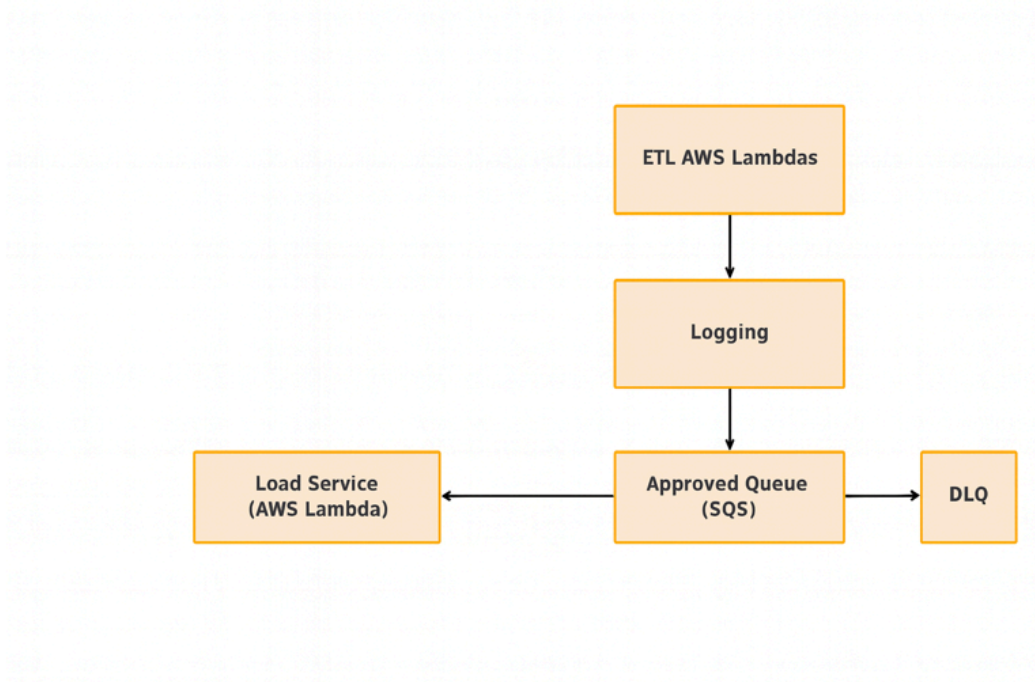
Goal – Configure and launch a single **Amazon SQS queue** called `ApprovedQueue`, which will act as the central buffer for all **approved catalog changes** (Products, Stock, Models, IPLs) that are ready to be synchronized with Shopware or other systems.

This queue enables:

- Asynchronous, decoupled delivery of syncable entities
- Improved reliability and traceability of approved data
- Scalable message consumption via Lambda functions

Access Control & IAM

- Producer: ETL Lambda services
- Consumer: Lambda service that performs sync
- IAM Policies:
 - UI role → `SendMessage`
 - Sync processor role → `ReceiveMessage`, `DeleteMessage`
- Visibility Timeout: Configurable based on expected processing time



List of tasks

#	Task	Description	Estimation, h	Jira	Notes
1	Create SQS queue	Define and provision SQS queue via Terragrunt Acceptance criteria: - ApprovedQueue exists in AWS - Queue name is consistent with naming conventions	8h	🔗 PAR-211: Create SQS queue DONE	
2	Define event structure	Specify the structure of messages placed on the queue Acceptance criteria: - Message contract is documented - Consumers can deserialize it	4h	🔗 PAR-212: Define event structure DONE	
3	Integrate ETL Lambdas with SQS queue	Modify the ETL Lambda Functions to automatically approve and publish approved messages to the queue Acceptance criteria: - ETL Lambdas trigger message publication with automatically approved changes - Messages appear in ApprovedQueue	6h	🔗 PAR-213: Integrate ETL Lambdas with queue DONE	
4	Implement consumer	Create Lambda-based service that reads from the queue and pushes data to Shopware Acceptance criteria: - Consumer reads and processes messages correctly - Data appears in Shopware	6h	🔗 PAR-214: Implement consumer DONE	

		• Duplicate messages are handled correctly • Failed messages are logged and sent to DLQ			
5	Configure IAM roles	Set up IAM permissions for producer to publish and sync processor to consume Acceptance criteria: • Producer has SendMessage access • Consumer has ReceiveMessage and DeleteMessage • Roles are scoped to the queue only	4h	🔗 PAR-215: Configure IAM roles DONE	
6	Create DLQ for failed events	Add SQS DLQ to capture undelivered/failed events for diagnostics Acceptance criteria: • DLQ is provisioned and attached to each SQS queue • Delivery attempts and failure threshold are configured • Messages move to DLQ after N retries • DLQ visible in AWS Console and accessible by diagnostics team	4h	🔗 PAR-216: Create DLQ for failed events DONE	

[Eliminated] Tech Design: Lambdas for load service

Overview

Goal - launch the service that receives approved changes from SQS and pushes them to Shopware.

This is the final component of the synchronization module.

Shopware Admin API: [Shopware Admin API](#) | [Admin API](#)

List of tasks

#	Task	Description	Estimation, h	Jira	Notes
1.	Setup repo	Create a new repo for a .NET Lambda project. Acceptance criteria: • The “catalog-load-function“ repo is created • The <i>master</i> and <i>development</i> branches are created • The <i>master</i> branch is protected • The <i>.gitignore</i> is added	2h		
2.	Create a Terraform module	Create a module “catalog-load-lambda“ that consists of the following resources: • <code>aws_lambda_function</code> - main resource defines Lambda • <code>aws_lambda_permission</code> - permission for Event Bridge to trigger Lambda Acceptance criteria: • Pull request with the module is merged into the <i>main</i> branch	4h		
3.	Create a Terragrunt unit	Create a unit from the module created at p2.	4h		

		Acceptance criteria: The unit is deployed manually on dev environment			
4.	Setup a project	Create a .NET AWS Lambda Function project. Use project templates: Building Lambda functions with C# - AWS Lambda Acceptance criteria: - Structured logging added (Serilog, CloudWatch) - AWS Secrets Manager is integrated - Lambda is deployed manually	4h		
5.	Setup CI/CD	Set up a CI/CD pipeline for the repo created at p1. Acceptance criteria: - static analysis is run, - unit and integration tests is run, - lambda built and deployed on dev	16		
6.	Implement loading	Based on event comes from SQS, reflect the change to Shopware using corresponding Admin API endpoints. Acceptance criteria: approved changes is visible in Shopware Admin	40		
7	IPL custom entity plugin	IPLs as Custom Entities in Shopware		PAR- 251: I mplem ent IP L cust om ent ity plu gin DONE	

CDN - Approach - [Outdated]

Details

- We don't implement new CDN for Shopware we should use the CDN that current Catalog uses.
- While ongoing sync process we:
 - a. For Models: Match media urls with Category.DisplayImage (model image)
 - b. For Parts: Match media urls with Category.DisplayImage (part image)
 - c. For IPLs: Combine 'file.checksumget' & 'file.size' fields to receive Media URL and match it with cdn_url property of IPL custom entity
- There is no FE work that should be Done. Everything supposed to work out of the box
- We need to implement separate logic for media files that we manually upload in ShopWare Admin to put it to the same CDN - [☑ PAR-423: Implement CDN usage for media that manually uploaded via Shopware Admin](#) TO DO

Custom Fields

List of Custom Fields

Type - Dev	Tech name - Dev	Type - Staging	Tech name - Staging	Set	Note
[Number]	custom_model_popularity	[Number]	custom_model_popularity	Model custom fields	CH: this is needed for PAR-425; may need to update to text field based on how PT stores their popularity field today
[Entity Select]	custom_model_marketing_brand	[Entity Select]	custom_model_marketing_brand	Model custom fields	
[Text]	custom_model_number	[Text]	custom_model_number	Model custom fields	
[Text]	custom_model_search_keywords	[Text]	custom_model_search_keywords	Model custom fields	
[Entity Select]	custom_model_cross_sells	[Entity Select]	custom_model_cross_sells	Model custom fields	
[Entity Select]	custom_model_attachments	[Entity Select]	custom_model_attachments	Model custom fields	
[Text]	custom_model_cutting_width	[Text]	custom_model_cutting_width	Model custom	

				fields	
[Text]	custom_model_age	[Text]	custom_model_age	Model custom fields	
[Text]	custom_model_engine_size	[Text]	custom_model_engine_size	Model custom fields	
[Text]	custom_model_handle_type	[Text]	custom_model_handle_type	Model custom fields	
[Text]	custom_model_power_type	[Text]	custom_model_power_type	Model custom fields	
		[Text]	custom_model_shaft_type	Model custom fields	CH: needs to be added to dev and included in part ingestion – needed for search filters
		[Text]	custom_model_trottle_location	Model custom fields	CH: needs to be added to dev and included in part ingestion – needed for search filters
		[Text]	custom_model_blade_type	Model custom fields	CH: needs to be added to dev and included in part

					ingestion – needed for search filters
		[Text]	custom_model _attachment_ty pe	Model custom fields	CH: needs to be added to dev and included in part ingestion – needed for search filters
[Number]	custom_model_varia nt_popularity	[Number]	custom_model _variant_popul arity	Model Variant custom fields	
[Text]	custom_model_varia nt_serial_range	[Text]	custom_model _variant_serial_ range	Model Variant Custom Fields	
[Text]	custom_model_varia nt_year	[Text]	custom_model _variant_year	Model Variant Custom Fields	
[Text editor]	custom_part_extra_d escription	[Text editor]	custom_part_e xtra_descriptio n	Part Custom Fields	
		[Checkbox]	custom_part_h azardous	Part Custom Fields	CH: needs to be added to dev and included in part ingestion – may be used

					in part UI and for rules
[Select Field]	custom_part_availability_code	[Select Field]	custom_part_availability_code	Part Custom Fields	
[Entity Select]	custom_part_replaced_by_part	[Entity Select]	custom_part_replaced_by_part	Part Custom Fields	
[Number]	custom_part_sales_velocity	[Number]	custom_part_sales_velocity	Part Custom Fields	
[Text]	custom_part_part_stock_id	[Text]	custom_part_part_stock_id	Part Custom Fields	
[Number]	custom_part_usually_ships_from	[Number]	custom_part_usually_ships_from	Part Custom Fields	
[Number]	custom_part_usually_ships_to	[Number]	custom_part_usually_ships_to	Part Custom Fields	
[Select Field]	custom_part_type	[Select Field]	custom_part_type	Part Custom Fields	CH: this may need to change to just text vs selection and included in part ingestion – needed for search filters
	custom_meta_category_type			custom_meta	

	custom_meta_surrogate_model_parent			custom_meta	
	custom_images_cdn_url		custom_images_cdn_url	custom_images	Duplication?
		Tax Exempt State(s) [Select Field]	custom_customer_fields_tax_exempt_states	Customer Custom Fields	CH: not needed; was using this to test approaches for managing tax exemption
		Form upload [Media Field]	custom_customer_fields_form_upload	Customer Custom Fields	CH: not needed; was using this to test approaches for managing tax exemption
		I am tax exempt [Checkbox]	custom_order_fields_tax_exempt	Order Custom Fields	CH: not needed; was using this to test approaches for managing tax exemption

		Note to PartsTree [Text]	custom_order_fields_CSR_note	Order Custom Fields	CH: will probably be needed (among others) as we implement checkout
		USPS Ships [Checkbox]	custom_addresses_fields_USPS_ships	Address Custom Fields	CH: can ignore these address custom fields; was using this to test approaches for building Flow Builder rules based off results of carrier API results
		FedEx Ships [Checkbox]	custom_addresses_fields_fedex_ships	Address Custom Fields	
		UPS Ships [Checkbox]	custom_addresses_fields_ups_ships	Address Custom Fields	
		Is this a PO Box? [Checkbox]	custom_addresses_fields_po_box	Address Custom Fields	
	custom_order_carrier_note		custom_order_carrier_note	Order Custom fields	

	custom_order_po_job_number		custom_order_po_job_number	Order Custom Fields	
	custom_customer_fields_default_carrier_note		custom_customer_fields_default_carrier_note	Customer Custom Fields	

Mapping doc [DocumentDB-Shopware data mapping](#)

Custom Fields mentioned in doc	Exists in Shopware(Y/N), Technical name	Entity	Comments
CustomFields[source]	???, custom_catalog_source	models	will be replaced to separate model_source
CustomFields[url]	Y???, custom_images_cdn_url	models	Needs discussion about attachment to set(s). Should it be common custom field for part and models or different fields (V.M) Were decided to use separate set for each partstree entity
CustomFields[search_keywords]	Y, custom_model_search_keywords	models	
CustomFields[equipment_type]	Y, seems we have duplication here: custom_family_fields_equipment_type	models	CH: my preference would be to keep "custom_family_fields..." version to maintain the convention of including the entity in the field name, but either works VM: ok Parameter should be attached to model parent

CustomFields[popularity]	Y, discuss - custom_model_popularity - custom_model_variant_popularity	models	CH: can consolidate to “custom_model_popularity”, the two fields serve the same function for models & variants, respectively. VM: needs discussion. We’re not sure in purpose of this parameter Were decided that we should keep both. Still unclear what to migrate. Will be clarified later
CustomFields[short_name]	Y, custom_model_short_name	models	CH: Victor, agree there isn’t a need to remove it if we’re already populating it. Currently not used in the UI. VM: after discussion with Igor we think that non-usable parameters should be removed. Needs discussion Confirmed
CustomFields[serial_range]	Y, custom_model_variant_serial_range	models	
CustomFields[year_range]	Y, custom_model_variant_year	models	
CustomFields[purchased_together]	Y, custom_model_cross_sells	models	CH: I probably mis-marked somewhere. The technical name is “custom_model_cross_sells”, and the human-facing label is “Purchased Together”
CustomFields[cutting_width]	Y, custom_model_cutting_width	models	
CustomFields[model_number]	Y, custom_model_model_number	models	

CustomFields[attachments]	Y, custom_model_attachments	models	
CustomFields[shaft_type]	Y, custom_model_shaft_type	models	
CustomFields[model_age]	Y, custom_model_model_age	models	
CustomFields[power_type]	Y, custom_model_power_type	models	
CustomFields[handle_type]	Y, custom_model_handle_type	models	
CustomFields[trottle_location]	Y, custom_model_trottle_location	models	CH: Newly added in staging based on Dima's findings. Needs to be added to Dev. Used for search filtering.
CustomFields[blade_type]	Y, custom_model_blade_type	models	
CustomFields[attachment_type]	Y, custom_model_attachment_type	models	
CustomFields[partstree_source]	Y???, custom_catalog_source	Parts	Needs discussion about attachment to set(s). Should it be common custom field for part and models or different fields (V.M) described above
CustomFields[partstree_type]	N	Parts	CH: don't need [partstree_type] as mentioned in the doc (maps to cataloger "type" field); "custom_part_part_type" should

			map to cataloger [partstree_part_type] / “part_type” field) V.M: the row will be removed
CustomFields[partstree_url]	Y???, custom_images_cdn_url	Parts	Needs discussion about attachment to set(s). Should it be common custom field for part and models or different fields (V.M) Described above
CustomFields[partstree_availability_code]	Y, custom_part_availability_code	Parts	
CustomFields[partstree_stockid]	Y, custom_part_pt_stock_id	Parts	
CustomFields[partstree_hazardous]	Y, custom_part_hazardous	Parts	
CustomFields[partstree_commodity_code]	N	Parts	CH: not needed, but don't need to remove either. Per PT, all their parts have the same commodity code. VM: after discussion with Igor, we think that all unnecessary fields should be removed. Needs discussion removal Confirmed
CustomFields[partstree_usually_ships_x]	Y, custom_part_usually_ships_from	Parts	CH: let's use custom_part_usually_ships_from as it's more clear VM: needs discussion and confirmation. There is an array in catalog. it means that there might be >2 values Confirmed, it's a range. If >2 values - it's bad data

CustomFields[partstree_usually_ships_y]	Y, custom_part_usually_ships_to	Parts	CH: let's use custom_part_usually_ships_to as it's more clear VM: needs discussion and confirmation. There is an array in catalog. it means that there might be >2 values Confirmed, it's a range. If >2 values - it's bad data
CustomFields[partstree_supersedes]	Y, custom_part_replaced_by_part	Parts	CH: correct technical field name is "custom_part_replaced_by_part"; maps to "supersedes" field in Cataloger
CustomFields[partstree_inserted]	N	Parts	CH: not needed VM: the row will be removed
CustomFields[[partstree_updated]	N	Parts	CH: not needed VM: the row will be removed
CustomFields[partstree_velocity]	Y, custom_part_sales_velocity	Parts	
CustomFields[partstree_part_type]	Y, custom_part_part_type	Parts	CH: see above re: partstree_type
CustomFields[partstree_brand_mfg]	should map to Product.Manufacturer.Name		
CustomFields[partstree_color_codes]	Should map to Product.Variants		

keywords: discussion, duplication, removed

Custom Fields - Name Convention

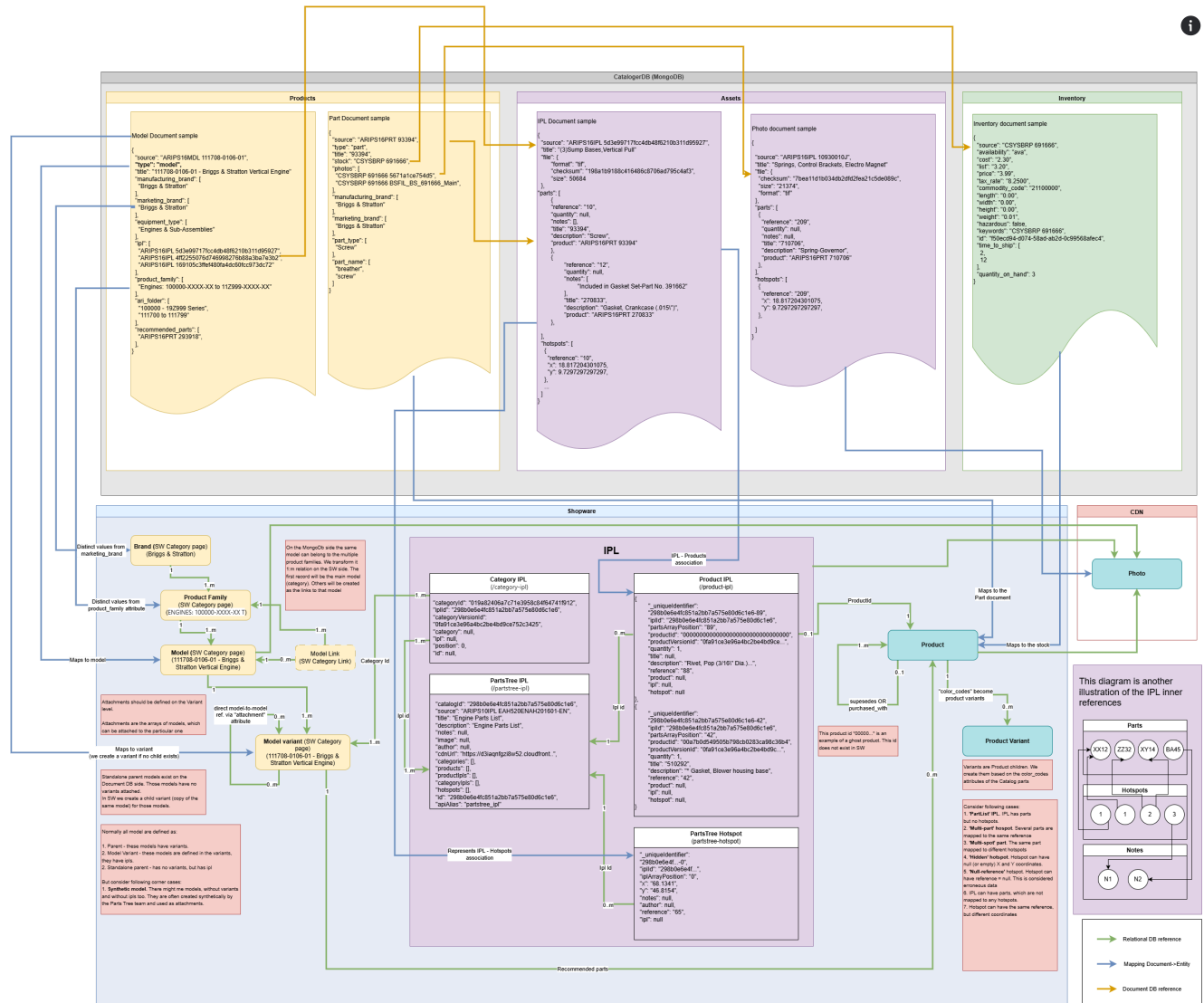
Field Sets:

1. A separate field set must be created for every business entity (brand, family, model parent, model variant, part, etc.), regardless of what Shopware entity it is.
2. A field set must have a name that matches the following template: `custom_<entity name>`, e.g., `custom_brand`, `custom_family`, `custom_model_parent`, etc.

Fields:

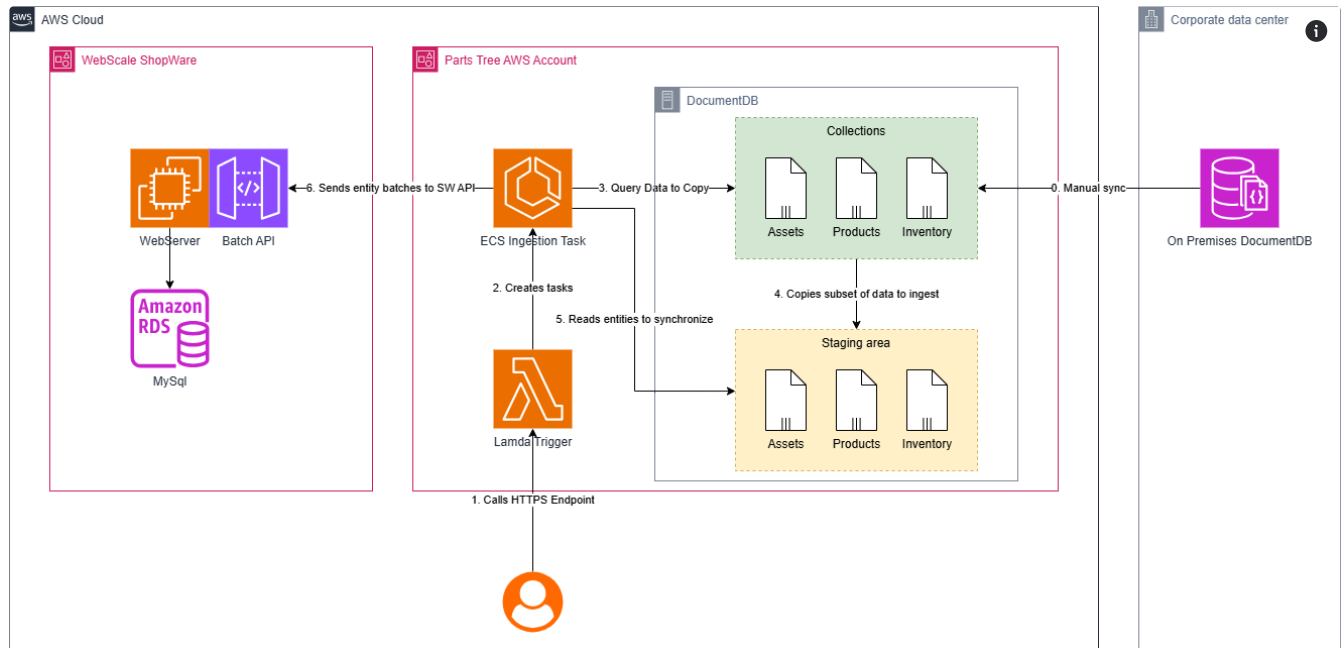
1. A field must have a name that matches the following template: `<field set name>_<field name>`, e.g., `custom_model_parent_source`, `custom_part_type`, etc.
2. The shortest possible field name should be chosen, as long as it remains meaningful and robust.

Cataloger DB to Shopware mapping



Data Ingestion Architecture

Current Architecture



Existing architecture capabilities

Production catalog DB to AWS replication - manual only.

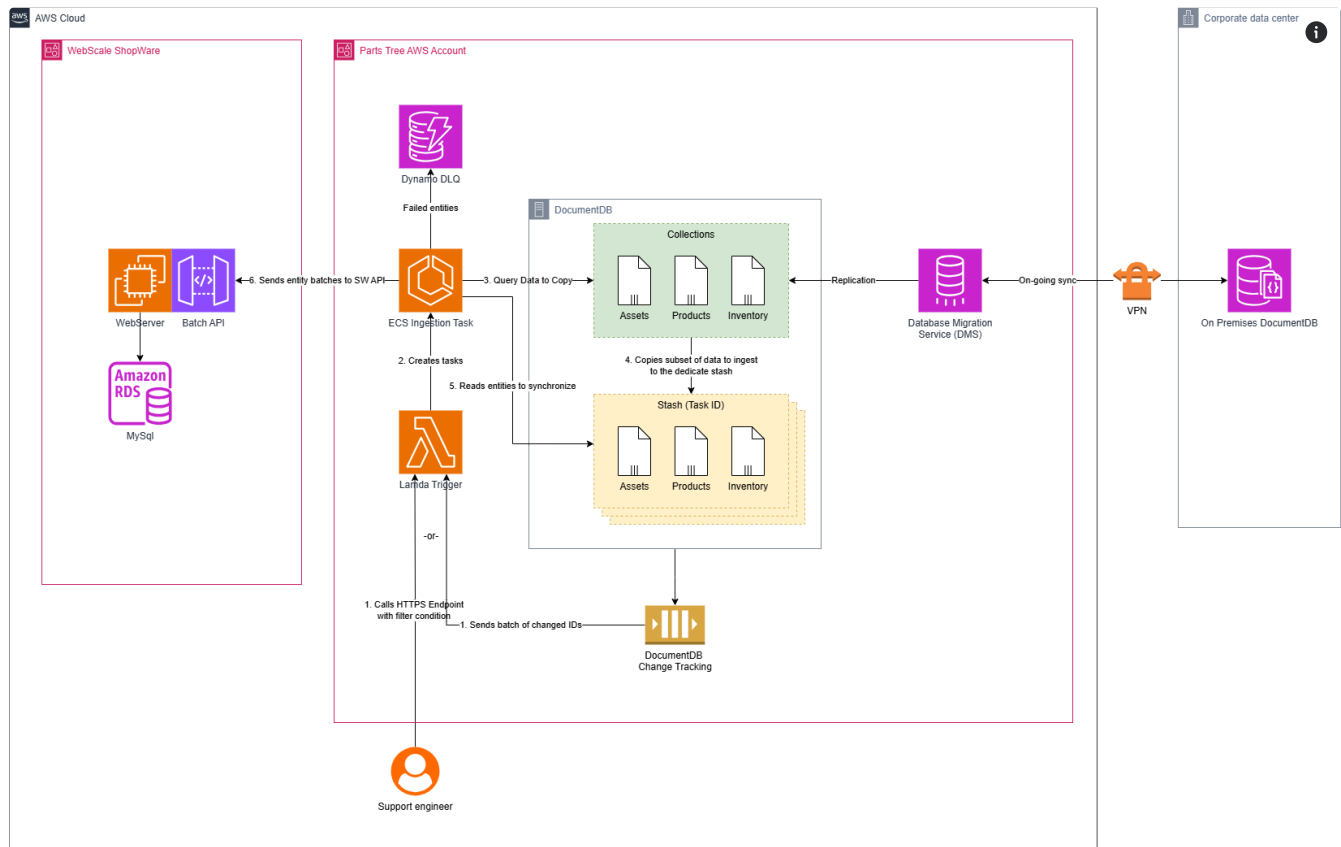
Task trigger - HTTPS endpoint. Supports entity type selection, brand selection, and other MongoDB-compatible filters.

Example

```
1 {
2   "batchSize": 1000,
3   "ingest": {
4     "brands": false,
5     "families": false,
6     "modelParents": false,
7     "modelVariants": false,
8     "ipl": false,
9     "manufacturers": false,
10    "parts": false,
11    "modelIpl": true,
12    "partIpl": false
13  },
14  "filter": {
15    "marketing_brand": "Gravelly"
16  }
17 }
```

Staging area - one instance, shared by all tasks. No concurrent synchronization tasks allowed.

Proposed architecture (full & partial sync)



New capabilities

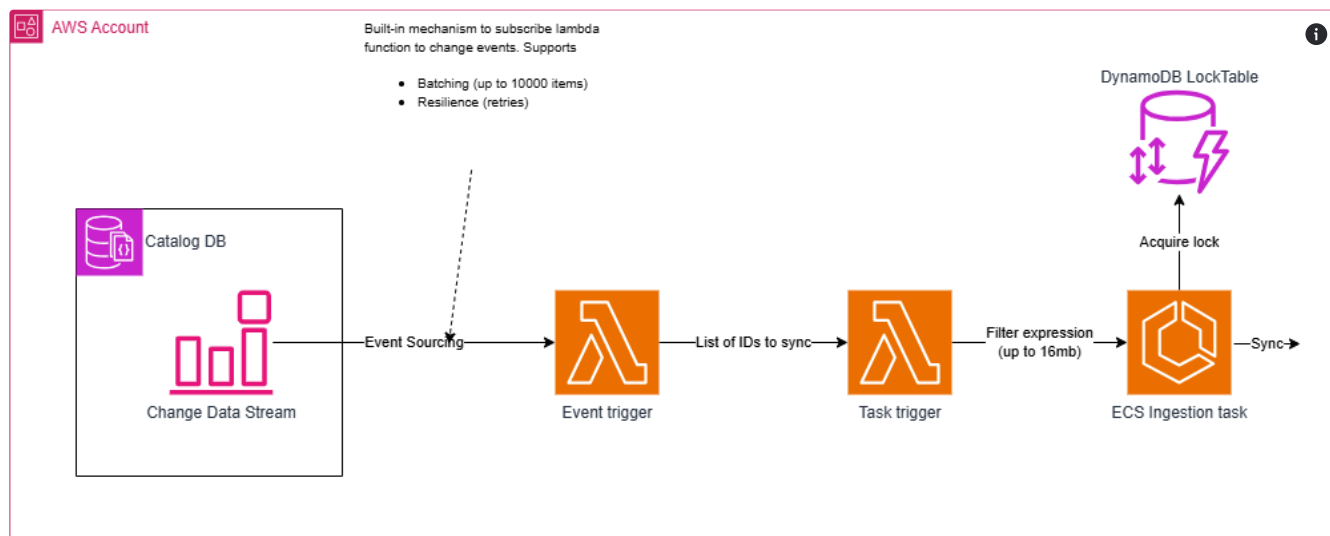
Ongoing synchronization of the production database to the AWS cloud via Database Migration Service

Staging/stash areas per task

Data change capture and task triggering for the list of entity IDs

Dead letter queue to capture and replay failed entities.

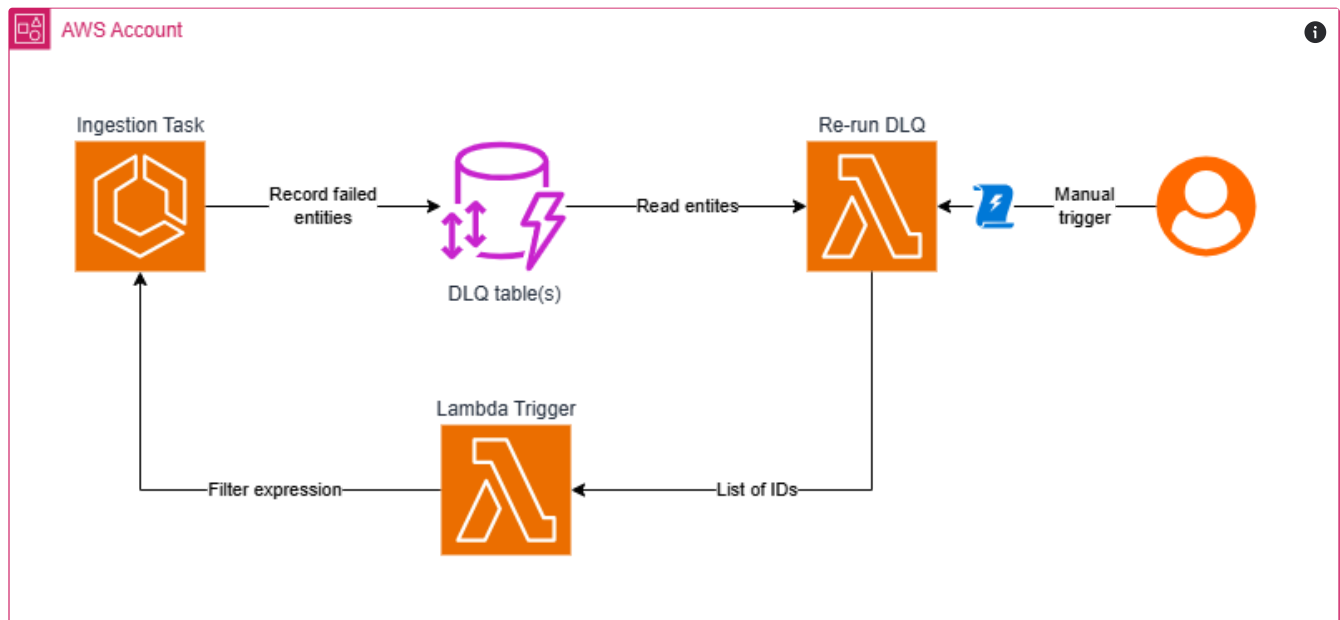
Change tracking and sync triggering



Required capabilities

1. Change data stream - built-in DocumentDb capability
2. Event sourcing/subscription for a lambda function - built-in lambda capability
 - a. Error handling/resilience - built-in retries
 - b. Batching - built-in lambda capability.
3. Event trigger. Collects IDs of the entities to sync and passes them to the Task trigger. Custom lambda function.
4. Task trigger - existing lambda function. Need to be enhanced to support
 - a. Accept a list of IDs for all types of entities/collections.
5. Concurrency management in the ECS tasks. Use a DynamoDB table for this goal.

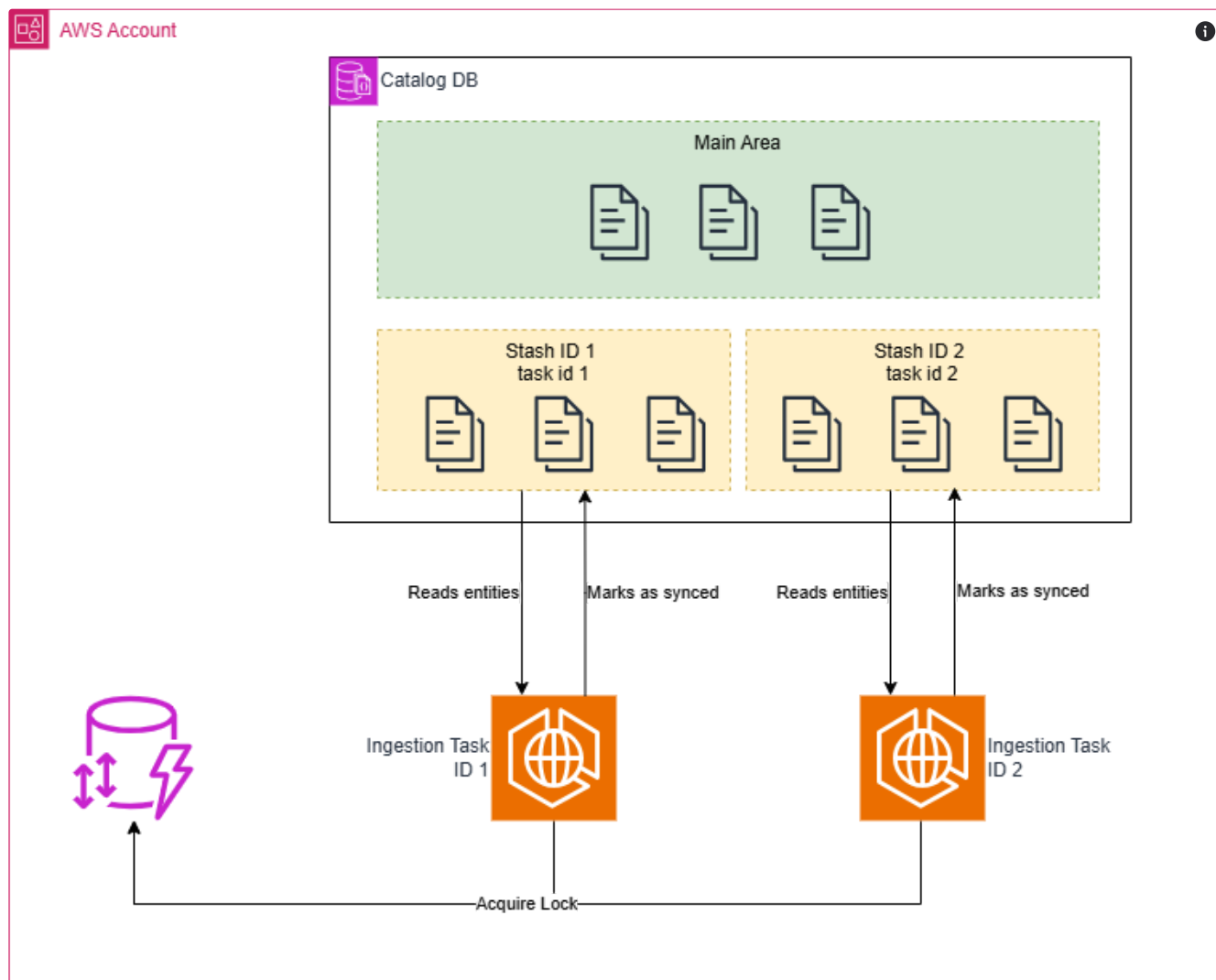
Task resilience and DLQ



Required capabilities

1. Dynamo DB DLQ table - stores failed entities and error details. Replaces current CloudWatch logs.
2. Re-run DLQ lambda. Simple script to define a slice and trigger a new task to synchronize

Task concurrency and re-enterability



Required capabilities

1. Dynamo DB Lock Table - the task must acquire a lock to start synchronization.
2. Multiple stashes - each task writes entities to sync into its own staging area.
3. Synchronization flag - mark entities in the staging area as synchronized as soon as SW confirms it.
4. Ability to re-run the task for the existing staging area (passing stash ID instead of the filter)

Ingestion verification approach

[todo]

Data Ingestion: Ongoing sync trigger

Goal: Support following client requirements for the ongoing sync

1. Catalog updates should be synced during off hours
2. Live operational data such as (stock, price etc) should be synced no more than 10 mins after it was received from the change stream.

Problems/Risks:

1. Frequent updates for live operational data from source (PartsTree). What if we will receive 40 small batches in a span of 3-5 mins?
2. Concurrency while appending data. While building a stage, task can be triggered multiple times in a row since change data stream has no guarantee on how the batch will get organized, we need to ensure that if we receive frequent updates we can trigger task multiple times to add data into stage either in concurrent mode or wait mode.
3. Concurrency while syncing data. When we trigger sync, either catalog or operational data we need to ensure that we still can process change stream data that is coming from the lambda.

Proposed solution

Option A (Not Viable, too much work):

Requirements

1. Tasks should be able to run in two distinct modes SYNC and APPEND
2. In append mode, it will accept new IDs for catalog and inventory updates and will keep building the stage for both
3. In Sync mode it will sync staged data with the Shopware.
4. Append mode might need to support greater level of concurrency or have an ability to queue tasks in order to handle frequent updates, this is more applicable to the Inventory updates rather than Catalog updates but ideally should work for both
5. While in sync mode task should lock stage that is being synced to ensure no new data is being added while sync will run.
6. In sync mode we still should be able to gracefully handle new changes that are coming via the data change stream. Example: when we run Inventory sync and we are keep getting more

and more records for inventory, we should start to build a new staging collection for the next sync run.

1. Lambda get's change stream via trigger from the document db for assets, products or inventory collections.
2. lambda will create two distinct correlation ID's for two types of the sync's. One for Live Operational data and one for the catalog data. (if correlation ID already exists it will reuse it.)
3. Lambda will write all event's into distinct dynamo collections, CATALOG_SYNC, LIVE_SYNC (names are subjects for discussion) with every record having same correlation id that have been generated before.
4. Lambda will pass data down to the worker with defined params such as: list of id's for all collections, run mode and correlation id
5. Task will receive data in form of event from lambda. Event sample

```
1 {  
2   "assets": {  
3     "ids": [1, 2, 3, 4],  
4     "correlationId": "catalog_123"  
5   },  
6   "products": {  
7     "ids": [1, 2, 3],  
8     "correlationId": "catalog_123"  
9   },  
10  "inventory": {  
11    "ids": [1, 2, 3],  
12    "correlationId": "live_123"  
13  },  
14  "mode": "append"  
15 }
```

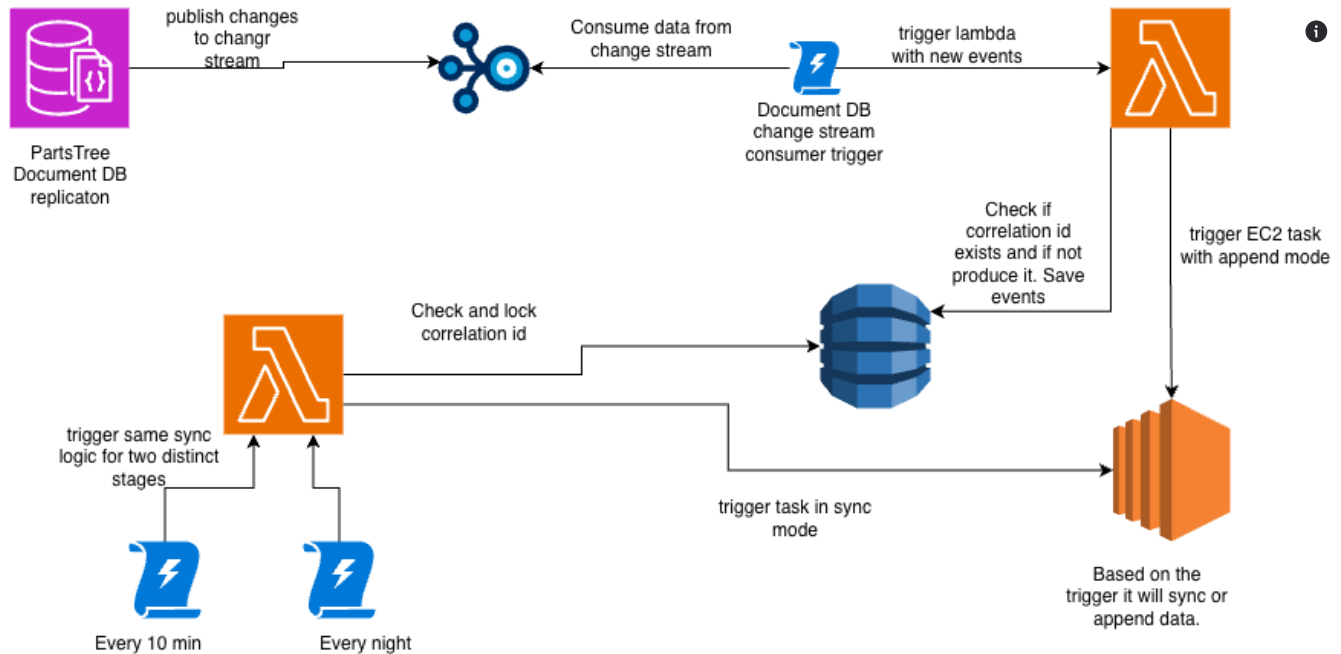
6. Once data is received task will build stages (or append to existing stage) by using the stage name + corid
7. New lightweight lambda is created to have two scheduled triggers every 10 minutes and at specific time.
8. Each trigger will have it's own flow once 10 minute one is triggered lambda will look at dynamo to get the correlationid for live data, mark it as locked and will send next event

```
1 {  
2   Mode: "Sync"  
3   Correlationid : "live_123"  
4 }
```

9. Once task is triggered, it will perform the sync of live data stage, will mark it as done(stage won't get deleted it will get renamed for future investigations etc) and in case if there is no errors will mark as completed DynamoDB data for the given correlationId

10. Same logic should apply for the Catalog sync, just with less frequent interval.

Potential issues: Between trigger to sync and trigger to receive data from mongo might be a time where sync lambda won't be able to get lock on the correlation id and new records might get appended to old correlation id, potential solution is limit lambda to 1 instance and in time trigger lambda check if there is current execution of the lambda if not then correlationID can be marked is locked if running then wait untill finished.



Option B: Collect & Release

This approach delegates collection responsibility to Lambda functions, which gather change events and store them for batch processing.

Architecture Flow

1. Change Stream Collection

- Lambda functions listen to DocumentDB change streams via triggers for the Assets, Products, and Inventory collections
- Changes are captured in real-time as they occur in MongoDB

2. Event Storage in DynamoDB

- Lambda writes all change events to DynamoDB tables with the following structure:
 - Partition Key: `id` (entity ID from MongoDB)
 - Sort Key: `timestamp`

- Attributes: `status` , event data
- Global Secondary Index (GSI) on `status` column to enable efficient querying by processing state
- Status values: `PENDING` , `PROCESSING` , `COMPLETED` , `FAILED`

3. Continuous Data Collection

- Lambda continues collecting changes for both Catalog and Operational data into DynamoDB
- All events are initially marked as `PENDING`

4. Scheduled Processing

- Trigger Lambda invokes the processing task on a schedule (every 10 minutes or end of day)
- This initiates the batch synchronization of collected data

5. Worker Claims and Processes Events

- Worker queries DynamoDB GSI for all `PENDING` records
- Retrieved records are immediately updated to `PROCESSING` status to claim them
- Worker processes each event and syncs to the destination system
- Successfully processed records are marked as `COMPLETED`

6. Concurrent Operations

- While the worker processes claimed events, new change events continue to be written with `PENDING` status
- These new events will be picked up in the next processing cycle

7. Failure Handling

- Implement logic to detect and mark failed synchronizations
- Failed events should be updated to `FAILED` status for retry or investigation
- Consider adding retry count and error message attributes

8. Events reconciliation:

- Write custom lightweight process to sweep all events during EOD and reconcile with what was processed
- sample

```

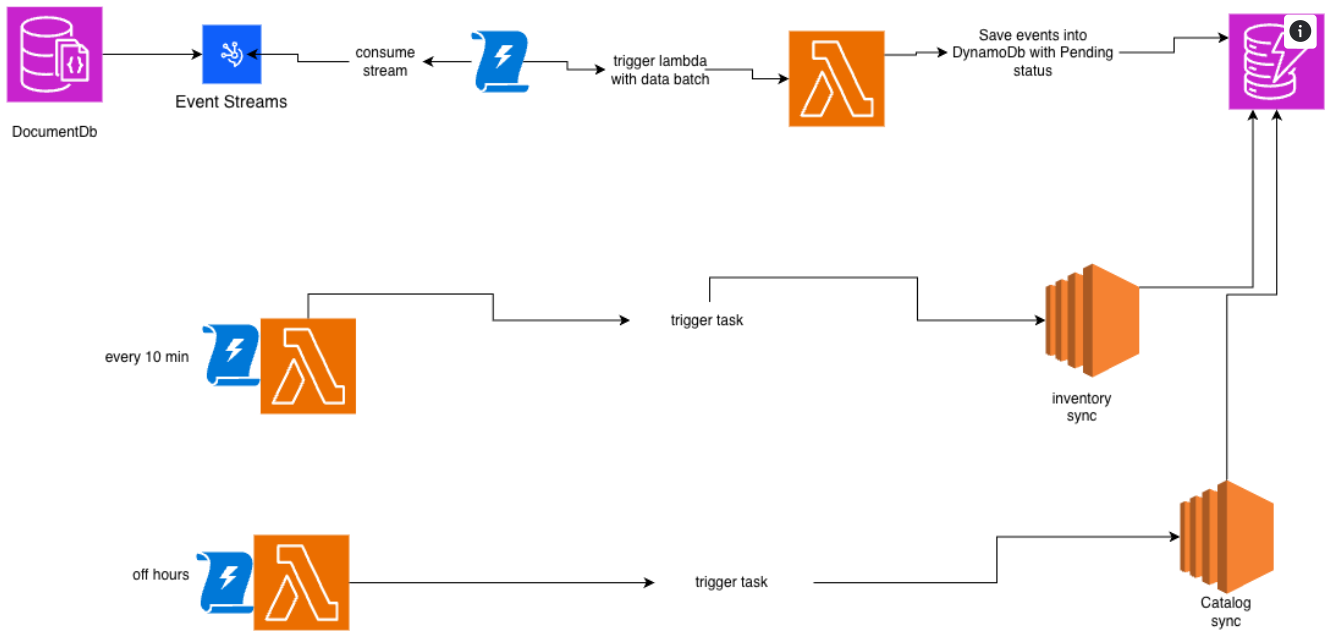
1  const oneWeekAgo = new Date();
2  oneWeekAgo.setDate(oneWeekAgo.getDate() - 7);
3
4  const startTimestamp = Timestamp(Math.floor(oneWeekAgo.getTime() / 1000), 0);
5  const collection = db.getCollection('products_streamtest');
6
7  const changeStream = collection.watch([], {
8    startAtOperationTime: startTimestamp
9  });
10
11  for (let i = 0; i < 1000; i++) {

```

```

12  const event = changeStream.tryNext();
13  if (!event) break;
14  printjson(event);
15  }
16
17  changeStream.close();

```



Refactoring MongoDB Data Pipeline for Improved Error handling

Summary

We need to refactor the way how we compose MongoDB data to be ready for sync with shopware in order to improve data quality, visibility and potentially apply preemptive fixes to data before running sync.

Goals

1. Remove MongoDB aggregation pipeline logic in favor of simple CRUD operations
2. Use domain models instead of BsonDocument/JsonObject for type safety
3. Introduce data sanity strategies to handle and log data quality issues
4. Add deduplication strategies to detect and prevent duplicate records

Current State vs. Proposed State

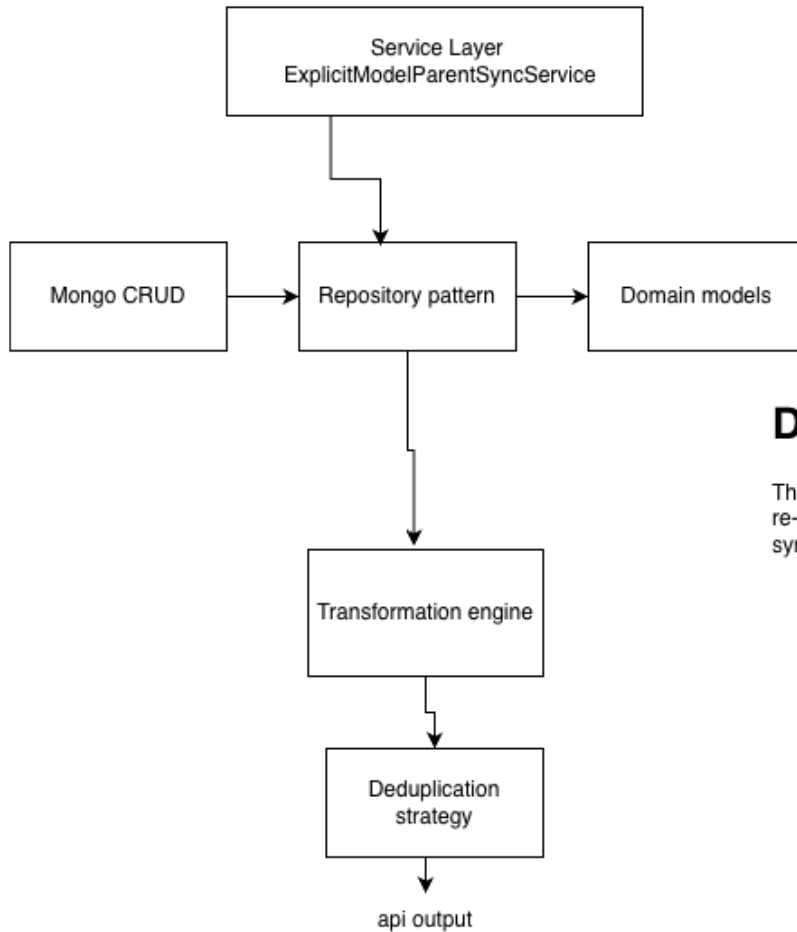
Aspect	Current State	Proposed State
Data Retrieval	Complex MongoDB aggregation pipeline with \$lookup, \$unwind, \$project	Simple Find() queries with filters
Data Models	BsonDocument → JsonObject mapping	Strongly-typed domain models (Product, Asset, CategoryPageDto)
Business Logic	Embedded in MongoDB pipeline stages	Transform engine with pluggable strategies
Data Quality	Hard-coded null coalescing in \$project	Explicit data sanity transforms with logging
Deduplication	None	Configurable deduplication strategies

Testability	Difficult to test aggregation logic	Easy to mock and unit test each component
-------------	-------------------------------------	---

Architecture Overview (NOTE: the code is pseudocode at this point and just outlines an approach or vision, coding details is down to developer choice)

Layer Separation

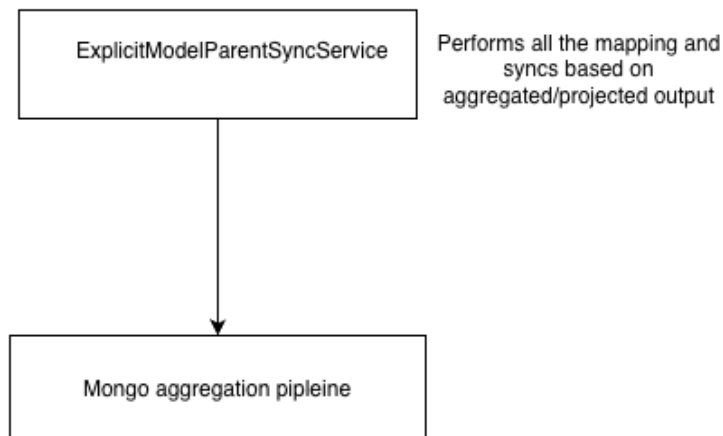
Proposed state



Dependecny sample

This diagram is the example of how we can re-arrange one of our services that perform sync of ExplicitModelParent

Current State



The architecture consists of three main layers:

- Service Layer: ExplicitModelParentSyncService orchestrates the data flow
- Data Access Layer: Repository Pattern provides simple MongoDB CRUD operations

- Business Logic Layer: Transform Engine and Deduplication Strategies handle data processing

These layers interact with:

- MongoDB CRUD: Simple queries replacing complex aggregations
- Domain Models: Strongly-typed C# classes (Product, Asset, CategoryPageDto)

Implementation Details

1. Domain Models

Instead of working with BsonDocument and JsonObject, we introduce strongly-typed models:

```

1 public class Product
2 {
3     public Guid Id { get; set; }
4     public string Source { get; set; }
5     public string Title { get; set; }
6     public List<string> MarketingBrand { get; set; }
7     public List<string> ProductFamily { get; set; }
8     public List<string> Photos { get; set; }
9     // ... other fields
10 }
11
12 public class Asset
13 {
14     public string Source { get; set; }
15     public AssetFile File { get; set; }
16 }
17
18 public class CategoryPageDto
19 {
20     public string Id { get; set; }
21     public string ParentId { get; set; }
22     public string Name { get; set; }
23     public CustomFieldsDto CustomFields { get; set; }
24 }

```

2. Repository Pattern (Simple CRUD)

```

1 public interface IProductRepository
2 {
3     Task<List<Product>> GetApprovedModelsAsync();
4     Task<List<Asset>> GetAssetsBySourcesAsync(IEnumerable<string> sources);
5 }
6
7 public class ProductRepository : IProductRepository
8 {
9     private readonly IMongoCollection<BsonDocument> _products;
10    private readonly IMongoCollection<BsonDocument> _assets;
11    private readonly IMapper _mapper; // AutoMapper
12
13    public async Task<List<Product>> GetApprovedModelsAsync()
14    {
15        // Simple filter instead of complex aggregation
16        var filter = Builders<BsonDocument>.Filter.And(
17            Builders<BsonDocument>.Filter.Eq("type", "model"),

```

```

18     Builders<BsonDocument>.Filter.Eq("meta.approved", true),
19     Builders<BsonDocument>.Filter.Eq("meta.deleted", false)
20 );
21
22     var docs = await _products.Find(filter).ToListAsync();
23     return _mapper.Map<List<Product>>(docs);
24 }
25
26 public async Task<List<Asset>> GetAssetsBySourcesAsync(IEnumerable<string> sources)
27 {
28     var filter = Builders<BsonDocument>.Filter.In("source", sources);
29     var docs = await _assets.Find(filter).ToListAsync();
30     return _mapper.Map<List<Asset>>(docs);
31 }
32 }

```

Benefits:

- Clear separation between data access and business logic
- Easy to mock for unit testing
- AutoMapper handles BsonDocument → Domain Model conversion

3. Transform Engine (Data Sanity Strategies)

```

1 public interface IFieldTransform<TSource, TTarget>
2 {
3     bool CanTransform(TSource source, TTarget target);
4     void Apply(TSource source, TTarget target);
5 }
6
7 public class TransformEngine<TSource, TTarget> where TTarget : new()
8 {
9     private readonly List<IFieldTransform<TSource, TTarget>> _transforms = new();
10
11     public TransformEngine<TSource, TTarget> Register(IFieldTransform<TSource, TTarget>
12     transform)
13     {
14         _transforms.Add(transform);
15         return this;
16     }
17
18     public TTarget Transform(TSource source)
19     {
20         var target = new TTarget();
21         foreach (var transform in _transforms.Where(t => t.CanTransform(source, target)))
22         {
23             transform.Apply(source, target);
24         }
25         return target;
26     }
27 }

```

Example Transforms

String Length Validation with Logging:

```
1 public class StringLengthTransform : IFieldTransform<Product, CategoryPageDto>
2 {
3     private readonly int _maxLength;
4     private readonly ILogger _logger;
5
6     public bool CanTransform(Product source, CategoryPageDto target)
7         => source.Title?.Length > _maxLength;
8
9     public void Apply(Product source, CategoryPageDto target)
10    {
11        _logger.LogWarning(
12            "Title truncated for product {ProductId}: '{Title}' ({Length} chars)",
13            source.Id, source.Title, source.Title.Length);
14
15        target.Name = source.Title.Substring(0, _maxLength);
16    }
17 }
```

Image URL Validation with Logging:

```
1 public class ImageUrlTransform : IFieldTransform<Product, CategoryPageDto>
2 {
3     private readonly ILookup<string, Asset> _assetLookup;
4     private readonly ILogger _logger;
5
6     public void Apply(Product source, CategoryPageDto target)
7     {
8         foreach (var photo in source.Photos ?? Enumerable.Empty<string>())
9         {
10             var assets = _assetLookup[photo].ToList();
11
12             if (!assets.Any())
13             {
14                 _logger.LogWarning("No asset found for photo {PhotoId} in product {ProductId}",
15                     photo, source.Id);
16                 continue;
17             }
18
19             // Build URLs for valid assets only
20         }
21     }
22 }
```

4. Deduplication Strategies

```
1 public interface IDeduplicationStrategy<T>
2 {
3     IEnumerable<T> Deduplicate(IEnumerable<T> items);
4 }
5
6 Strategy Options
7
```

```

8 Batch-Level Deduplication:
9
10 public class BatchDeduplicationStrategy : IDeduplicationStrategy<CategoryPageDto>
11 {
12     private readonly ILogger _logger;
13     public IEnumerable<CategoryPageDto> Deduplicate(IEnumerable<CategoryPageDto> items)
14     {
15         var duplicates = items.GroupBy(x => x.Id).Where(g => g.Count() > 1);
16
17         foreach (var group in duplicates)
18         {
19             _logger.LogWarning("Found {Count} duplicates for ID {Id} in batch",
20                 group.Count(), group.Key);
21         }
22
23         return items.GroupBy(x => x.Id).Select(g => g.First());
24     }
25 }
26
27 Cross-Batch In-Memory Deduplication:
28
29 public class InMemoryDeduplicationStrategy : IDeduplicationStrategy<CategoryPageDto>
30 {
31     private readonly HashSet<string> _seenIds = new();
32     private readonly ILogger _logger;
33     public IEnumerable<CategoryPageDto> Deduplicate(IEnumerable<CategoryPageDto> items)
34     {
35         return items.Where(item => {
36             if (_seenIds.Add(item.Id))
37                 return true;
38
39             _logger.LogWarning("Duplicate ID {Id} detected across batches", item.Id);
40             return false;
41         });
42     }
43 }

```

5. Main Service Orchestration

```

1 public async Task SyncExplicitPrimaryParents()
2 {
3     // 1. Simple CRUD queries
4     var products = await _repository.GetApprovedModelsAsync();
5     var photoIds = products.SelectMany(p => p.Photos).Distinct().ToList();
6     var assets = await _repository.GetAssetsBySourcesAsync(photoIds);
7     var assetLookup = assets.ToLookup(a => a.Source);
8
9     // 2. Setup transform engine with pluggable strategies
10    var engine = new TransformEngine<Product, CategoryPageDto>()
11        .Register(new BasicFieldsTransform())
12        .Register(new StringLengthTransform(255, _log))
13        .Register(new ImageUrlTransform(assetLookup, _log));
14
15    // 3. Process with deduplication
16    foreach (var batch in products.Chunk(_batchSize))
17    {
18        var transformed = batch.Select(p => engine.Transform(p));
19        var deduplicated = _deduplicationStrategy.Deduplicate(transformed);
20
21        await _categories.BulkUpsert(deduplicated);
22    }

```

Configuration Examples

```

1 No Deduplication:
2
3 var service = new ExplicitModelParentSyncService(
4     products, assets, categories, cdnUrl, log,
5     new NoDeduplicationStrategy<CategoryPageDto>());
6
7 Batch-Level Only:
8
9 var service = new ExplicitModelParentSyncService(
10     products, assets, categories, cdnUrl, log,
11     new BatchDeduplicationStrategy(log));
12
13 Full Stack (Composite):
14
15 var service = new ExplicitModelParentSyncService(
16     products, assets, categories, cdnUrl, log,
17     new CompositeDeduplicationStrategy(
18         new BatchDeduplicationStrategy(log),
19         new InMemoryDeduplicationStrategy(log),
20     ));

```

Benefits Summary

Benefit	Description	
Maintainability	Business logic in C# is easier to understand and modify than MongoDB aggregations	
Testability	Each transform and strategy can be unit tested independently	
Type Safety	Compile-time checking prevents runtime errors from typos or type mismatches	
Observability	Explicit logging in transforms helps identify data quality issues	
Flexibility	Strategies can be swapped without changing core logic	

Migration Strategy

1. Create domain models and repository layer
2. Setup AutoMapper profiles for BsonDocument → Domain Model mapping
3. Implement transform engine and initial transforms
4. Run both pipelines in parallel, compare outputs
5. Add deduplication strategies based on observed duplicate patterns
6. Switch traffic to new pipeline
7. Remove old aggregation code

Open Questions

- Should we use AutoMapper or manual mapping for BsonDocument conversion?
- What's the acceptable memory footprint for in-memory deduplication?
- Should transforms be configurable via appsettings.json?

DocumentDB-Shopware normalization → validation → fixing mapping

Source

 [DocumentDB-Shopware normalization → validation → fixing mapping](#)

Table column reference

Column	Description
Business field	Human-readable business attribute represented by the row. This is the conceptual field being traced across the pipeline, for example: Brand, Part Number, Price, Weight, Image Checksum.
Root collection	The top-level MongoDB collection from which the row originates in the pipeline context. For example, <code>products</code> , <code>inventory</code> , <code>assets</code> .
Source field(s)	Original field or fields from the documented database that supply the value. Use the documented path notation, for example <code>manufacturing_brand</code> , <code>inventory.price</code> , <code>assets.file.checksum</code> . If multiple fields participate, list all of them.
Join / relationship	Describes how the value is obtained when it does not come directly from the root document. This column should explicitly show the join key path and relationship, for example <code>products.stock -> inventory.source</code> or <code>products.photos[] -> assets.source</code> . For array expansion, indicate <code>\$unwind</code> as well.
Transformation semantics	Short description of what happens to the value at this step of the flow. Examples: <code>direct pass-through</code> , <code>lookup</code> , <code>lookup + unwind</code> , <code>trim</code> , <code>enum mapping</code> , <code>type coercion</code> , <code>unit conversion</code> , <code>deduplication</code> , <code>fallback</code> , <code>truncate</code> . This column explains the nature of the transformation without showing implementation details.
Projected field	The field name as it appears after the aggregation/project mapping stage. This is the actual output shape produced by the pipeline, for example <code>stockObject.price</code> or <code>photoAssets.file.checksum</code> .
Normalized field	The internal application field used after part normalization. This should point to the normalized model property, for example <code>Price</code> or <code>BrandName</code> .

Validation	Validation rules applied to the normalized value. Examples: required, nullable, max length, numeric range, enum membership, valid URL, non-negative quantity. This column explains what must be true for the row to remain valid.
Fixing	Automatic correction or normalization logic that may be applied before the value is rejected. Examples: convert units, uppercase, remove duplicates, apply fallback value. If no fixing is allowed, state <code>none</code> . Applies only if validation fails
Shopware payload	The target field or structure in the final Shopware payload. This should show the final destination of the business attribute, for example <code>product.name</code> , <code>product.stock</code> , <code>product.weight</code> , <code>product.customFields.*</code> , manufacturer association, media association, and so on.
Source DB type	Source data type as stored or documented in the database. Examples: <code>string</code> , <code>int</code> , <code>decimal</code> , <code>bool</code> , <code>array<string></code> , <code>object</code> , <code>array<object></code> . This should reflect the documented source shape, not the normalized application type.
App type	Final internal application type after normalization and parsing. Examples: <code>string</code> , <code>decimal</code> , <code>int</code> , <code>bool</code> , <code>array<string></code> , <code>PartImageModel</code> . This column helps identify type conversions and mapping risks.
Edge cases	Special handling, non-obvious behavior, or edge cases associated with the row. Examples: lookup may return no match, array is unwound into multiple rows, value may be null but preserved, source vocabulary differs from Shopware vocabulary, one source record may generate multiple output records.
Aggregation request (separate in json)	Actual DocumentedDB aggregation pipeline used to retrieve the source dataset for the table.

TODO: Write about normalization rules

How to use the table

1. Read each row as one traceable business attribute

Each row should represent one meaningful attribute, not just one raw field name.

For example:

- Price
- Brand
- Part Number
- Image checksum

2. Follow the row from left to right

The table should be read as a transformation chain:

documented DB source

→ **aggregation / projection result**

→ **normalized model**

→ **validation / fixing**

→ **Shopware payload**

This allows the reader to understand:

- where the value originally came from,
- how it was transformed,
- where it ended up.

3. Use source paths, not vague names

In `Source field(s)` and `Join / relationship`, always use explicit field paths.

Good:

- `direct`
- `<collection-name> lookup key`
- `products.stock = inventory.source`
- `products.photos[] = assets.source`

Bad:

- `stock lookup`
- `photo join`

The goal is to make origin and relationship unambiguous.

4. Be explicit for array expansion

If a source array is unwound, state that clearly in `Transformation semantics`.

? - preserve null/empty

Examples:

- `unwrap + map`
- `unwrap? + map`
- `map array`
- `lookup + unwrap? + map`

This helps explain why one source document can produce multiple output rows or multiple payload candidates.

5. Separate validation from fixing

Validation answers: **what is allowed**.

Fixing answers: **what can be automatically corrected**.

Example:

- **Validation:** not empty, max length 255
- **Fixing:** empty as default, truncate to max length, delete empty values

6. Use `Edge cases` for non-obvious behavior only

This column should not repeat validation rules.

Use it only for important exceptions or pipeline-specific behavior, such as:

- lookup may not find a matching row,
- null is preserved by `unwrap`,
- one record may generate multiple outputs,
- source value may use legacy enum vocabulary,
- field is internal-only and not always sent to Shopware.

7. Prefer business clarity over implementation detail

The table should explain the mapping clearly, not reproduce code literally.

It is acceptable to summarize implementation details as long as the data lineage remains exact.

For example:

- `parse decimal`
- `normalize brand casing`
- `map legacy availability to Shopware status`

is better than dumping internal method names.

8. Update the table whenever the mapping contract changes

The table should be treated as a mapping contract reference.

It must be updated when any of the following changes:

- source field paths,
- lookup relationships,
- normalization property names,
- validation rules,
- fixing behavior,
- Shopware payload structure.

If code changes but the table does not, the document becomes misleading very quickly.

Recommended conventions

Path notation

Use dot notation for nested fields:

- `photoAssets[].file.checksum`
- `stockObject.quantity_on_hand`

Use `[]` for arrays:

- `products.photos[]`
- `photoAssets[]`

Join notation

Use equals notation for joins:

- `products.stock = inventory.source`
- `products.photos[] = assets.source`

Type notation

Use compact readable types:

- `string`
- `bool`
- `int`
- `decimal`

- `array<string>`
- `object`
- `array<object>`

? - nullable reference type

! - can empty, but not nullable

Empty values

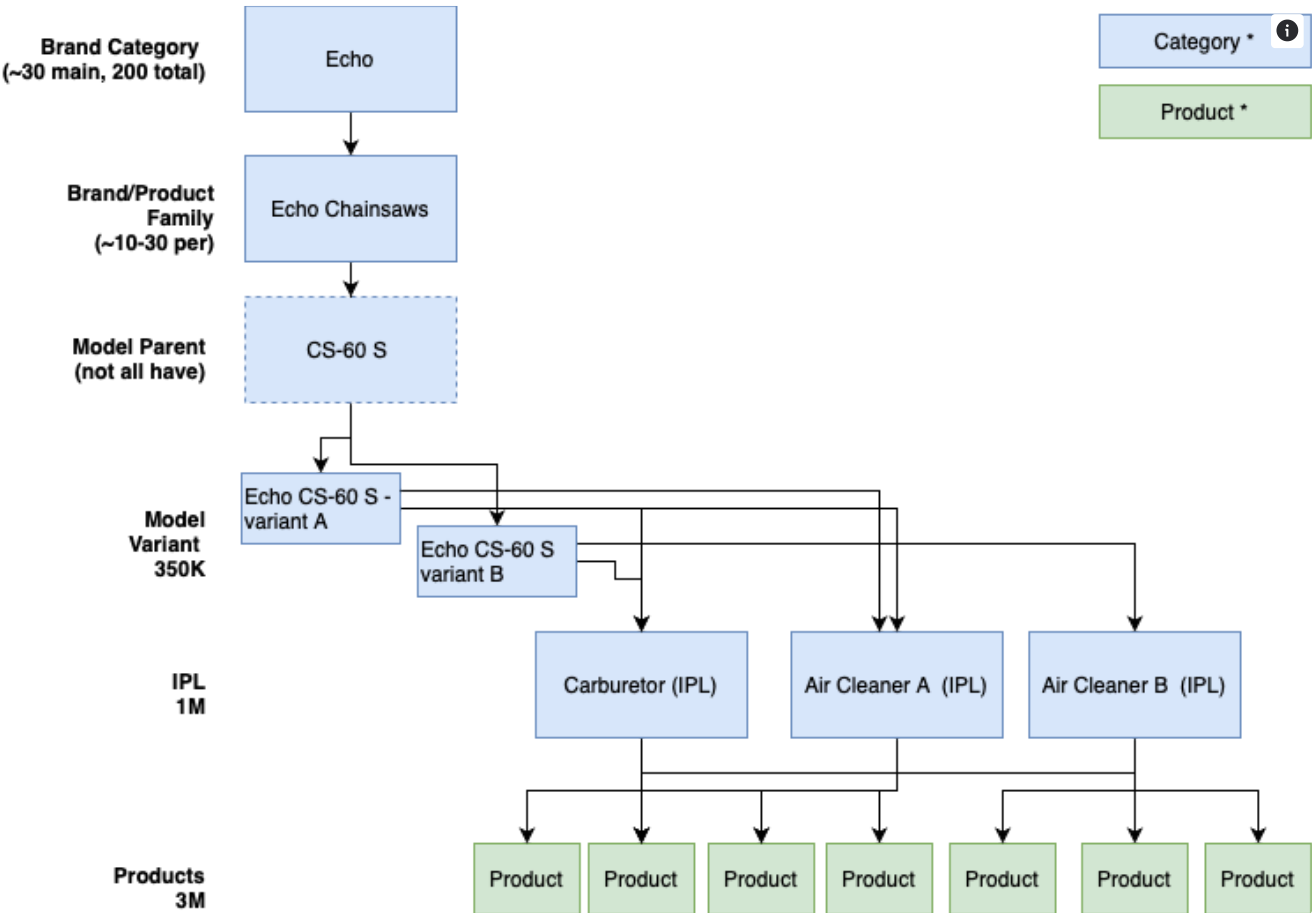
When a column does not apply, prefer explicit markers:

- `n/a`

instead of leaving cells blank.

Catalog Hierarchy

Existing:



Solutions:

